



INDIVIDUAL ASSIGNMENT - PART 2

ADVANCED SOFTWARE SECURITY

CT123-3-3-ASC

SECURE SOFTWARE IMPLEMENTATION REPORT

BLOOD DONATION SYSTEM

INTAKE CODE: APD3F2601CS(CYB)

LECTURER / TUTOR: NOR AZLINA BINTI ABD RAHMAN

DATE ASSIGNED: 9 June 2026

DATE COMPLETED: 20 September 2026

WEIGHTAGE: 60%

NAME: HEMANTHRAJAH A/L N KRISHNARAJAH

TP NUMBER: TP070844

Report scope: Secure system design, STRIDE threat modelling, secure coding implementation and post-remediation security testing.

Table of Contents

List of Figures	3
List of Tables	6
Abstract	7
1.0 Introduction	7
1.1 Implementation Scope and Environment	8
1.2 Implementation Objectives	9
2.0 Secure System Design and Threat Modelling	9
2.1 Secure Architecture and Trust Boundaries	10
2.2 STRIDE Threat Model	11
2.3 Security Requirements and Mitigation Coverage	11
3.0 Secure Coding Implementation and Justification	12
3.1 Staff Login SQL Injection, Password Security and MFA	12
3.2 Donor Search SQL Injection	18
3.3 Plaintext Password Storage and Database Migration	19
3.4 RBAC and Operational Report Data Minimisation	20
3.5 Predictable Donation Pass / Insecure Direct Object Reference	24
3.6 Stored Cross-Site Scripting	26
3.7 Cross-Site Request Forgery Protection	26
3.8 Session Security and Security Response Headers	29
3.9 Safe Error Handling, Logging and Additional Hardening	30
4.0 Secure Testing Results and Analysis	32
4.1 Semgrep	33
4.2 Bandit	33
4.3 OWASP ZAP	34
4.4 sqlmap	35
4.5 pip-audit	36
4.6 pytest Security Regression Testing	36
4.7 Nikto	37
4.8 Consolidated Secure Testing Result	37
5.0 Residual Risk and Limitations	38
6.0 Conclusion	39
Appendix A Additional Implementation Evidence	41
7.0 References	50

List of Figures

Figure 1: Secure architecture, trust boundaries and primary security controls.....	10
Figure 2: Parameterized staff-login query and password-hash verification in the secure version	13
Figure 3: Login rate handling and pending-MFA session isolation.....	14
Figure 4(a): Manager password accepted but MFA still required.....	14
Figure 4(b): Dashboard available only after MFA completion.....	14
Figure 4(c): Google Authenticator TOTP code for the manager01 account	15
Figure 5: TOTP MFA setup and QR-code generation.....	16
Figure 6: Atomic TOTP verification with time-step replay prevention.....	17
Figure 7(a): Authenticator enrollment screen.....	17
Figure 7(b): MFA enabled for the manager account.....	17
Figure 8: Parameterized donor-search query in the secure version.....	18
Figure 9(a): Vulnerable search broadened the returned records	18
Figure 9(b): Secure search treats the injection-style value as search data.....	18
Figure 10: Secure staff schema with password_hash, MFA state and security audit log	19
Figure 11: Replay-safe database migration for the secure schema	20
Figure 12: Server-side RBAC helpers and denied-access audit behaviour.....	21
Figure 13: Manager/admin report authorization and data-minimised query	22
Figure 14(a): Nurse dashboard without privileged report access	23
Figure 14(b): Operational report available to an authorized manager.....	23
Figure 15: Restricted CSV export with audit logging and formula neutralisation	24
Figure 16: Cryptographically random donation-pass token generation and storage.....	25
Figure 17: Server-side verification of the secure donation-pass token	26
Figure 18: Jinja auto-escaping used for stored health-screening content.....	26
Figure 19: Global CSRF protection and safe CSRF/error handlers.....	27
Figure 20: CSRF tokens embedded in state-changing staff forms	28
Figure 21(a): Secure donor registration form.....	28
Figure 21(b): Missing-CSRF request rejected with HTTP 400.....	28
Figure 22: Environment-based Flask secret and hardened session-cookie configuration	29
Figure 23: Security headers applied to application responses	30
Figure 24: ZAP evidence of the hardened session cookie in the secure application.....	30

Figure 25: Parameterized security audit logging helper.....	31
Figure 26(a): Login attempt and rate-limit helpers.....	32
Figure 26(b): Screening measurement validation.....	32
Figure 27(a): Finite decimal parser rejecting NaN/infinite input	32
Figure 27(b): Controlled dependency update check.....	32
Figure 28: Semgrep secure-project scan reporting zero findings	33
Figure 29: Bandit result for the fixed application - four Low, zero Medium/High findings.	34
Figure 30(a): ZAP - fixed login rejects SQL injection	34
Figure 30(b): ZAP - fixed donor search handles injection-style input safely	34
Figure 31(a): ZAP confirms CSRF token in the secure form	35
Figure 31(b): ZAP resend without CSRF token is rejected.....	35
Figure 32(a): Password success redirects to MFA verification.....	35
Figure 32(b): Invalid MFA code is rejected.....	35
Figure 33(a): sqlmap does not confirm injection in the improved login	36
Figure 33(b): sqlmap does not detect injection in the improved donor search	36
Figure 34(a): pip-audit identified two vulnerable dependencies before update	36
Figure 34(b): Dependency update completed and pip-audit reports no known vulnerabilities	36
Figure 35: Full regression suite after dependency updates - 40 passed and no known package vulnerabilities.....	37
Figure 36(a): Security-specific pytest run passed.....	37
Figure 36(b): Python compile verification completed successfully.....	37
Figure 37(a): Nikto baseline - missing anti-clickjacking header	37
Figure 37(b): Nikto secure scan - security headers present.....	37
Appendix Figure A1: TOTP time-step validation helper.....	41
Appendix Figure A2: MFA disable flow requiring re-authentication.....	42
Appendix Figure A3: Controlled library update checker.....	43
Appendix Figure A4: Login rate-limit regression test.....	43
Appendix Figure A5: TOTP replay and audit regression tests.....	44
Appendix Figure A6: Secure application home page.....	45
Appendix Figure A7: Secure staff login interface.....	45
Appendix Figure A8: Browser evidence that SQL injection login is rejected.....	46
Appendix Figure A9: Normal donor search as an authorized nurse	46
Appendix Figure A10: Secure operational report after role and data-minimisation changes	47

Appendix Figure A11: Code evidence for MFA provisioning secret and QR generation.....	48
Appendix Figure A12: Code evidence for MFA code verification.....	48
Appendix Figure A13: MFA-related database fields	48
Appendix Figure A14: Registered donors page after secure changes	49

List of Tables

Table 1: Secure implementation architecture and control overview..... 8

Table 2: STRIDE threat analysis and implemented mitigations..... 11

Table 3: Security requirements and mitigation coverage..... 12

Table 4: Consolidated secure testing results..... 38

Table 5: Residual risks and recommended future improvements..... 39

Abstract

Part 1 exposed a fairly broad set of weaknesses in the Flask/PostgreSQL Blood Donation System; this report records what changed once those weaknesses were actually fixed. The work covers the three areas required for Part 2: secure system design and threat modelling, implementation and justification of secure coding, and analysis of the final security tests. The original build suffered from SQL injection vulnerabilities in the staff logon and donor search, stored staff passwords in plaintext, provided too much information in reports, had predictable donation-pass references, allowed stored XSS, no CSRF protection, weak session settings, and technical detail in debug/error handling. Those issues are fixed in the revised build, using parameterized SQL, Werkzeug script password hashing, Google Authenticator-compatible TOTP MFA, server-side role checks, data minimisation, high-entropy donation-pass token, Jinja escaping, Flask-WTF CSRF protection, stronger session settings, response security headers, safer exception handling, rate controls, audit logging, and tight validation. These risks are associated with STRIDE: spoofing, tampering, repudiation, information disclosure, denial of service and elevation-of-privilege. The final system is then tested again with Semgrep, Bandit, OWASP ZAP, sqlmap, pip-audit, pytest and Nikto. What is important is not that every tool gives a perfect score; it's that the behaviours that were once exploitable no longer reproduce, the security regression tests pass and the remaining limitations are exposed and not treated as absolute secure.

1.0 Introduction

The baseline was established in Part 1. That plan was needed to weather the storm of the real application in part 2. A code diff alone tells very little; the new system must also have "failsafes" that prevent access to requests that previously succeeded, role boundaries that prevent a user from going directly to a protected route, and test results that are consistent with the intended design. According to NIST, "Secure Development" is defined in NIST SP 800-218 as a set of practices integrated into the software development lifecycle with a focus on reducing vulnerability and addressing their root causes, not on merely adding patches to them. It was this idea that guided the implementation. The design decision, code changes and repeatable tests were all considered to be the same exercise.

It remains a local, flask and PostgreSQL application, and is still only used in an approved academic lab. Donors register information and medical staff sign in to check the records and perform operational functions. The data is not simple, of course, and it's all behind the same application: personal identifiers, screening data, staff accounts, and blood-bank workflow data. The primary security objectives were thus confidentiality, integrity, trusted authentication, least privilege, and helpful audit trails. They are very similar to OWASP Top 10:2025 categories like Broken Access Control, Security Misconfiguration, Injection, and Authentication Failures and Mishandling of Exceptional Conditions is directly relevant to the previous debug and verbose-error behaviour (OWASP Foundation, 2025a).

1.1 Implementation Scope and Environment

The secure build was built and executed in local environment using a Windows/VS Code IDE. The original Linux version was kept separate to allow before/after testing to be done again without erasing the vulnerable baseline. The business workflow itself was intentionally retained: donor registration, staff login, donor search, screening updates, donation records, blood-bank queue functions, reporting and dependency checking are still retained. The changes have been in how those functions make security decisions. The screenshots and test results shown in this section are from the updated codebase and from evidence gathered during the implementation stage from the post-remediation results.

Component	Secure implementation
Application	Python Flask web application with server-side session handling
Database	PostgreSQL using psycopg2 parameter binding
Authentication	Username + bcrypt password verification; optional TOTP MFA
Authorization	Server-side login_required / role_required controls
Request integrity	Global Flask-WTF CSRF protection for state-changing requests
Output safety	Jinja auto-escaping; unsafe Markup rendering removed
Session protection	Environment secret, HttpOnly/SameSite settings, environment-controlled Secure flag
Monitoring	Security audit log for authentication, denials and sensitive exports
Verification	Semgrep, Bandit, ZAP, sqlmap, pip-audit, pytest and Nikto

Table 1: Secure implementation architecture and control overview

1.2 Implementation Objectives

- For all nine main vulnerabilities identified in Part 1 apply the mitigations where appropriate to the final build.
- Don't consider each vulnerability independently, instead link assets and trust boundaries with explicit security controls to perform a threat modelling.
- Eliminate the root issues that are causing the vulnerabilities, such as dynamic SQL, unsafe HTML rendering and other known vulnerabilities, instead of simply relying on filtering for known payloads.
- Add supporting controls like MFA, rate limiting, audit logging, security headers and tighter validation, and make sure these are differentiated from the main fixes.
- Perform the same attacks as previously against the updated system and observe the outcome and compare it with the vulnerable system.
- Record what is still imperfect. Residual risk is a part of the implementation story and should not be buried in silence.

2.0 Secure System Design and Threat Modelling

Threat modelling was done as a design process and not as an afterthought. That facilitated the assessment of the individual fixes being effective as a system. OWASP describes threat modelling as a structured approach for modelling an application and asking “what could go wrong?” in the application, and STRIDE is the list of six practical threat categories for conducting this analysis (OWASP Foundation, n.d.-a). Microsoft specifies them as Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, and Elevation of privilege (Microsoft, 2026). These are the most significant boundaries for this application: when potentially untrusted browser data gets into Flask, and when Flask gets into PostgreSQL / audit store.

2.1 Secure Architecture and Trust Boundaries

Secure Blood Donation System - Trust Boundaries and Data Flows

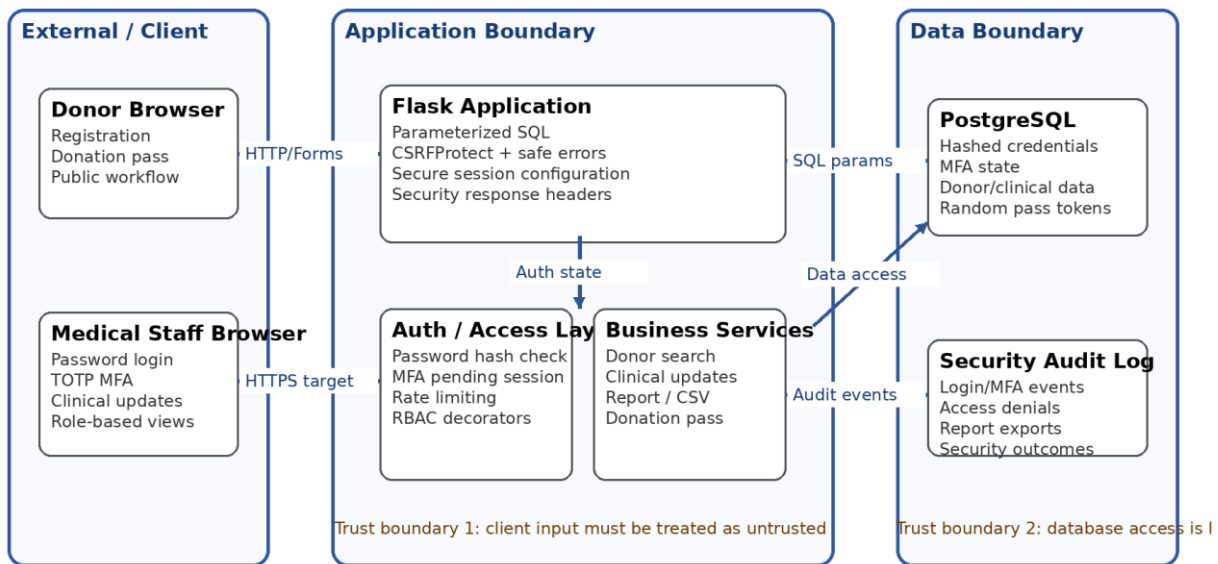


Figure 1: Secure architecture, trust boundaries and primary security controls

The first trust boundary is between the browser and the Flask app. If it's incoming from a donor or a staff browser, it is treated as untrusted, even if it was from an application's own form. This assumption is a direct route to bound SQL parameters, CSRF tokens on state-changing requests, escaped output, and split password authentication and MFA. The second layer is between application logic and persistent storage. In this case, queries are passed as parameters, staff credentials are stored as hash rather than readable passwords, report queries only return the fields that are relevant, and accessibility related actions are logged in a separate audit table. These decisions in practice are intended to prevent one coding error from becoming a whole system failure.

Authentication state is also intentionally acted out. When MFA is enabled, a correct password will establish a temporary PENDING-MFA state, but not a complete staff session. The transition is only completed with a valid TOTP. The same 'do not trust the interface' rule is also applicable to authorization. It is good user interface practice to hide an Operational Report button but it is not a security boundary. Even if the URL is typed, the route will again check with the user's role on the server.

2.2 STRIDE Threat Model

STRIDE category	Threats observed or plausible	Asset / boundary	Implemented mitigations
Spoofing	SQLi login bypass; stolen/reused staff password; replayed TOTP	Staff identity and authenticated session	Parameterized login, script hash verification, TOTP MFA, pending-MFA isolation, TOTP step replay prevention, login attempt controls
Tampering	SQL query manipulation; forged state-changing POST; manipulation of screening/donation data	Database queries and clinical workflow state	Parameterized queries, CSRFProtect, server-side validation, finite numeric parsing, role checks
Repudiation	User denies failed logins, access denials, report exports or MFA actions	Security-sensitive actions	security_audit_log, outcome field, staff/user context, request IP and timestamps
Information disclosure	Plaintext credentials, unrestricted operational report, predictable donation pass, verbose errors, weak cookies	Credentials, donor PII, screening records, session identifiers	Password hashes, RBAC, data minimisation, random tokens, safe errors, HttpOnly/SameSite cookies, security headers
Denial of service	Repeated login/MFA attempts; malformed numeric input; dependency-check process hanging	Authentication service and request processing	Rate limiting/lock handling, bounded validation, finite-number checks, subprocess timeout
Elevation of privilege	Nurse/doctor access to manager reporting; SQLi becoming staff session; direct object access	Role boundaries and privileged functions	role_required enforcement, deny-by-default checks, MFA before full session, token verification, audited denials

Table 2: STRIDE threat analysis and implemented mitigations

There's one thing the STRIDE review was very clear about: The nine findings outlined in Part 1 are overlapping. Login SQL injection starts out as a spoofing/authentication issue, but once bypassed, it can easily escalate into elevation of privilege and disclosure of information. The old Operational Report overlapped - it was too open to reveal information, due to the authorization model. Hence the implementation centered on trust boundaries and layered controls and not a neat, but impractical, 'one vulnerability, one patch' approach.

2.3 Security Requirements and Mitigation Coverage

ID	Security requirement	Applies to	Status
SR-01	All SQL using request-controlled data must be parameterized.	Login, donor search, audit insert	Implemented and retested
SR-02	Passwords must never be stored or compared as plaintext.	medical_staff authentication	password_hash + Werkzeug script
SR-03	Privileged staff can enable TOTP MFA; full session only after MFA success.	Authentication boundary	Implemented
SR-04	Sensitive routes must enforce authorization server-side.	Operational report/export	Manager/admin

			restriction
SR-05	Object references must not be predictable authorization secrets.	Donation pass	Random token + verification
SR-06	User-controlled content must remain encoded in HTML output.	Screening/notes fields	Jinja auto-escaping
SR-07	State-changing requests must carry valid CSRF tokens.	POST forms	Flask-WTF CSRFProtect
SR-08	Session cookies and secrets must use secure defaults.	Flask session	Environment secret, HttpOnly, SameSite
SR-09	Errors must fail safely and not expose SQL/stack details.	Global error paths	Generic handlers + server logging
SR-10	Security-relevant actions should be auditable.	Auth, access denial, report export	security_audit_log
SR-11	Known vulnerable dependencies must be reviewed and updated.	requirements.txt	pip-audit remediation

Table 3: Security requirements and mitigation coverage

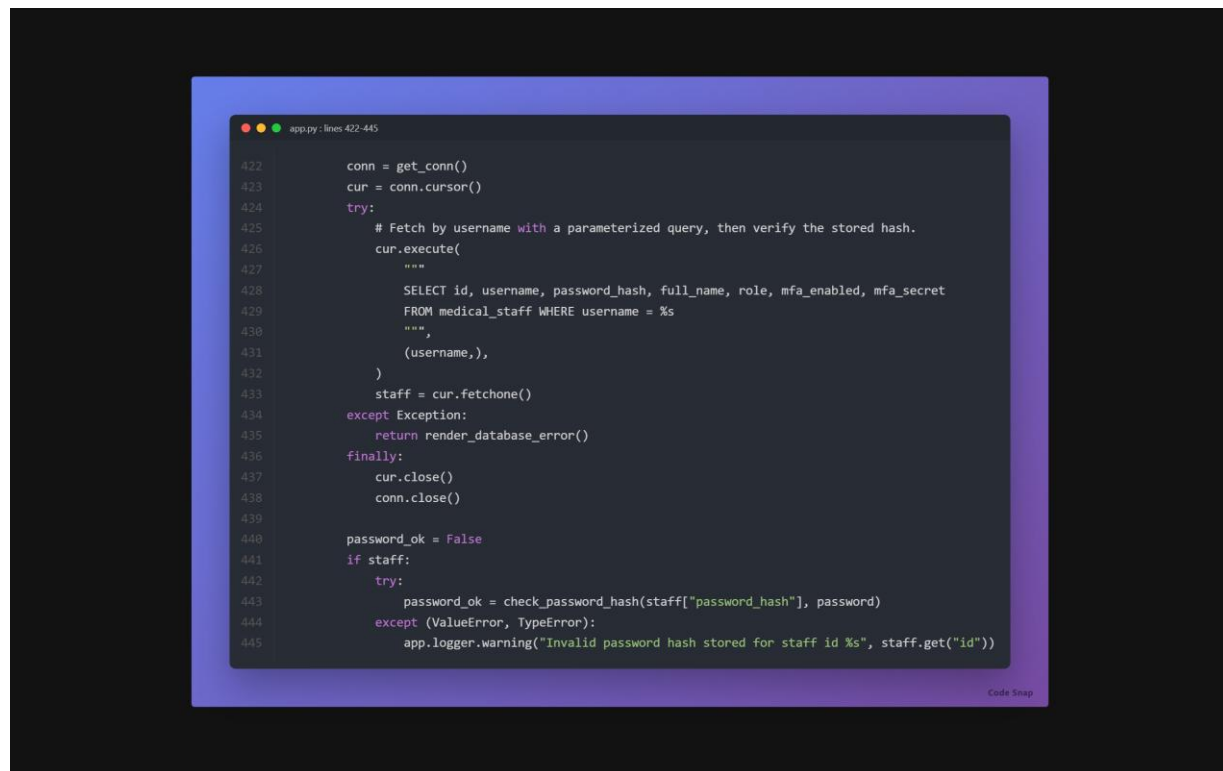
3.0 Secure Coding Implementation and Justification

This section details the modifications carried out for the nine key findings and additional reinforcement provided around the key findings. The discussion for each issue begins with the root cause, then presents the evidence for implementation, discusses the appropriateness of the control, and connects it to runtime verification. The basic principle was: if there's a decision to make about security, make it. Role checks are not simply checked through the navigation bar but through route logic, and SQL injection is eliminated because the SQL syntax and data are decoupled from each other, rather than attempting to blacklist quotes or other characters.

3.1 Staff Login SQL Injection, Password Security and MFA

The default username and password login was using the provided username and password in a literal SQL string. The username is passed into PostgreSQL as a bound parameter in the revised version, and thus treated as data, not as part of the statement itself. The returned account row is matched and the submitted password is compared to a hash using Werkzeug's `check_password_hash` function with `password_hash`. It matters that there is a separation: Modifying the username entered won't rewrite the SQL grammar again. That is why, according

to OWASP, prepared or parameterised statements are the recommended method of preventing SQL injection attacks (OWASP Foundation, n.d.-b).



```
422     conn = get_conn()
423     cur = conn.cursor()
424     try:
425         # Fetch by username with a parameterized query, then verify the stored hash.
426         cur.execute(
427             """
428             SELECT id, username, password_hash, full_name, role, mfa_enabled, mfa_secret
429             FROM medical_staff WHERE username = %s
430             """
431             (username,),
432         )
433         staff = cur.fetchone()
434     except Exception:
435         return render_database_error()
436     finally:
437         cur.close()
438         conn.close()
439
440     password_ok = False
441     if staff:
442         try:
443             password_ok = check_password_hash(staff["password_hash"], password)
444         except (ValueError, TypeError):
445             app.logger.warning("Invalid password hash stored for staff id %s", staff.get("id"))
```

Figure 2: Parameterized staff-login query and password-hash verification in the secure version

The problem with password hashing is that it won't just improve the cosmetic appearance of your login; it will minimize the impact if your database is ever breached. The updated schema has `password_hash` rather than a readable password. In this implementation (Pallets Projects, 2026a) `generate_password_hash` is implemented by default in Werkzeug 3.1 as `bcrypt` and `check_password_hash` provides one-way verification. OWASP likewise recommends modern adaptive password hashing and is explicit that plaintext password storage should not be used (OWASP Foundation, n.d.-c).

TOTP MFA was added after the primary authentication flaws had been removed. It is worth separating those ideas: MFA does not fix SQL injection; parameter binding does. What MFA adds is another barrier when a password is stolen or reused. With MFA enabled, a successful password check produces only a pending authentication state and redirects the user to the verification step. The normal staff session is created only after the TOTP is accepted.

```

447         if not staff or not password_ok or staff["role"] not in STAFF_ROLES:
448             locked = register_login_failure(username)
449             record_security_event(
450                 "staff_login",
451                 "locked" if locked else "failure",
452                 staff_id=staff.get("id") if staff else None,
453                 username=username,
454                 details="invalid_credentials",
455             )
456             flash("Invalid username or password.", "danger")
457             return render_template("staff_login.html"), 429 if locked else 401
458
459         clear_login_failures(username)
460
461         if staff["mfa_enabled"]:
462             # Do not create a privileged session until the second factor succeeds.
463             session.clear()
464             session["pending_mfa_staff_id"] = staff["id"]
465             session["mfa_attempts"] = 0
466             record_security_event(
467                 "staff_login",
468                 "success",
469                 staff_id=staff["id"],
470                 username=staff["username"],
471                 details="password_accepted_mfa_pending",
472             )
473             return redirect(url_for("mfa_verify"))
474
475         start_staff_session(staff)
476         record_security_event(
477             "staff_login",
478             "success",
479             staff_id=staff["id"],

```

Figure 3: Login rate handling and pending-MFA session isolation

Figure 4(a): Manager password accepted but MFA still required

Figure 4(b): Dashboard available only after MFA completion

During enrolment, the application generates a TOTP secret and an authenticator provisioning URI, presents the URI as a QR code, and waits for a valid code before switching MFA on. PyOTP documents the same provisioning and TOTP verification workflow for authenticator-compatible applications (PyOTP, n.d.). One extra control was added for replay: the last accepted TOTP time-step is stored, and the same step cannot be accepted twice. It is a small detail, but an important one, because a six-digit code can remain valid for the rest of its short time window.

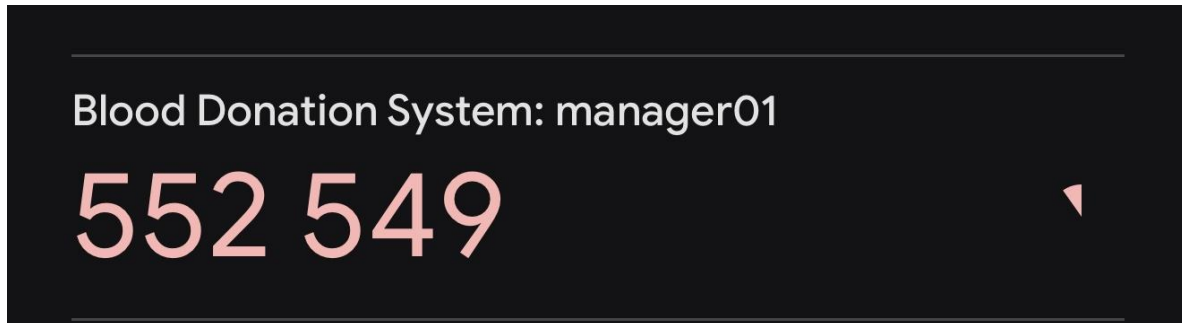


Figure 4(c): Google Authenticator TOTP code for the manager01 account

```

608     if not staff["mfa_enabled"] and not staff["mfa_secret"]:
609         staff["mfa_secret"] = pyotp.random_base32()
610         cur.execute(
611             """
612             UPDATE medical_staff
613             SET mfa_secret = %s, mfa_last_used_step = NULL
614             WHERE id = %s
615             """
616             , (staff["mfa_secret"], staff["id"]),
617         )
618         conn.commit()
619
620     if request.method == "POST" and not staff["mfa_enabled"]:
621         code = (request.form.get("code") or "").strip()
622         totp_step = matching_totp_step(staff["mfa_secret"], code)
623         if totp_step is not None:
624             cur.execute(
625                 """
626                 UPDATE medical_staff
627                 SET mfa_enabled = TRUE,
628                     mfa_confirmed_at = CURRENT_TIMESTAMP,
629                     mfa_last_used_step = %s
630                 WHERE id = %s
631                 """
632                 , (totp_step, staff["id"]),
633             )
634             conn.commit()
635             record_security_event(
636                 "mfa_setup",
637                 "success",
638                 staff_id=staff["id"],
639                 username=staff["username"],
640                 details="mfa_enabled",
641             )
642             flash("MFA has been enabled for your account.", "success")
643             return redirect(url_for("mfa_setup"))
644             record_security_event(
645                 "mfa_setup",
646                 "failure",
647                 staff_id=staff["id"],
648                 username=staff["username"],
649                 details="invalid_totp",
650             )
651             flash("Invalid or expired authenticator code. MFA was not enabled.", "danger")
652
653     if staff["mfa_enabled"]:
654         return render_template("mfa_setup.html", mfa_enabled=True)
655
656     uri = pyotp.TOTP(staff["mfa_secret"]).provisioning_uri(
657         name=staff["username"], issuer_name="Blood Donation System"
658     )
659     qr_image = qrcode.make(uri)
660     buffer = io.BytesIO()
661     qr_image.save(buffer, format="PNG")
662     qr_data = base64.b64encode(buffer.getvalue()).decode("ascii")
663     return render_template(
664         "mfa_setup.html",
665         mfa_enabled=False,
666         qr_data=qr_data,
667         manual_key=staff["mfa_secret"],
668     )

```

Figure 5: TOTP MFA setup and QR-code generation

```

522     cur.execute(
523         """
524         SELECT id, username, full_name, role, mfa_enabled, mfa_secret,
525                mfa_last_used_step
526         FROM medical_staff WHERE id = %s
527         """,
528         (staff_id,),
529     )
530     staff = cur.fetchone()
531     totp_step = matching_totp_step(staff["mfa_secret"], code) if staff else None
532     if staff and staff["mfa_enabled"] and totp_step is not None:
533         # The conditional update atomically claims this TOTP time-step. A
534         # captured code cannot be reused, even by two simultaneous requests.
535         cur.execute(
536             """
537             UPDATE medical_staff
538             SET mfa_last_used_step = %s
539             WHERE id = %s
540             AND (mfa_last_used_step IS NULL OR mfa_last_used_step < %s)
541             RETURNING id
542             """,
543             (totp_step, staff_id, totp_step),
544         )
545         verified = cur.fetchone() is not None
546         replayed = not verified
547         if verified:
548             conn.commit()
549         else:
550             conn.rollback()

```

Figure 6: Atomic TOTP verification with time-step replay prevention

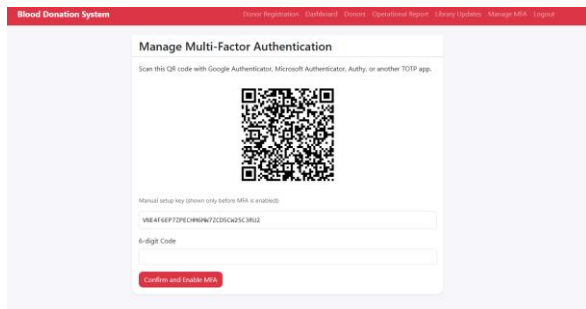


Figure 7(a): Authenticator enrollment screen

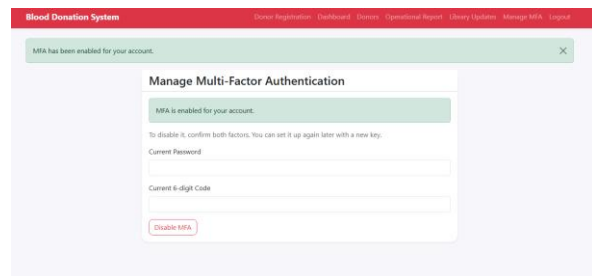


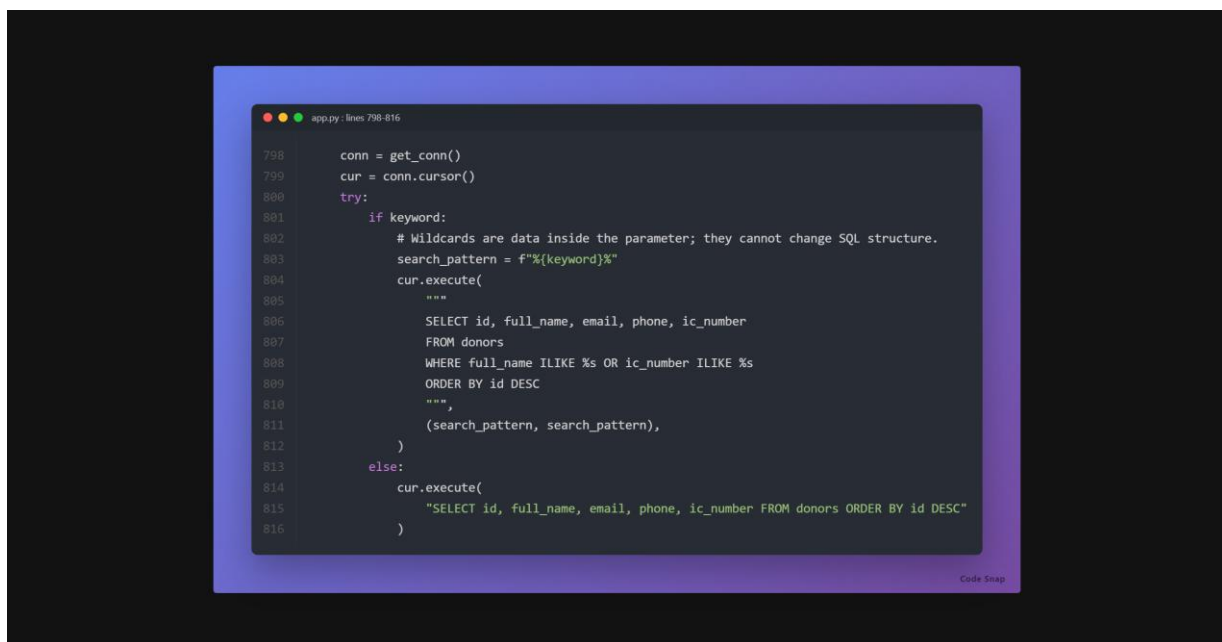
Figure 7(b): MFA enabled for the manager account

Repeated login attempts are handled separately from the core authentication check. These controls are not designed for password and MFA validation – they’re just designed to discourage brute forcer behaviour and leave more evidence behind. Failure will increase the temporary denial/lock behaviour and the audit helper will keep the result and request context. This provides

effective protection against the denial-of-service and repudiation vulnerabilities that are part of the STRIDE attack without altering the underlying login paradigm.

3.2 Donor Search SQL Injection

A SQL injection path was found on donor search. The old way of using the q value was to put it directly in an ILIKE statement, and making the login query wouldn't have resolved that either. In the updated path, the full-name and IC/passport to compare are bound into psycopg2 as part of the application code. A length check is still useful for robustness, but is intentionally used as a secondary measure. The removal of the injection condition is the parameter binding



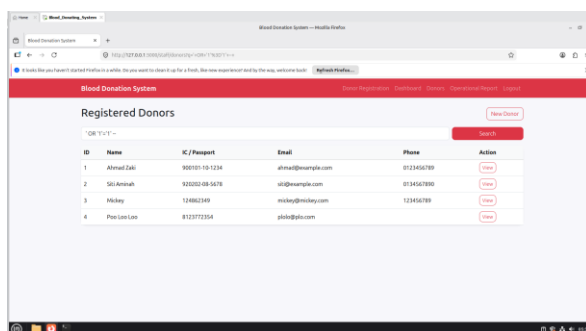
```

app.py: lines 798-816

798     conn = get_conn()
799     cur = conn.cursor()
800     try:
801         if keyword:
802             # Wildcards are data inside the parameter; they cannot change SQL structure.
803             search_pattern = f"%{keyword}%"
804             cur.execute(
805                 """
806                 SELECT id, full_name, email, phone, ic_number
807                 FROM donors
808                 WHERE full_name ILIKE %s OR ic_number ILIKE %s
809                 ORDER BY id DESC
810                 """,
811                 (search_pattern, search_pattern),
812             )
813         else:
814             cur.execute(
815                 "SELECT id, full_name, email, phone, ic_number FROM donors ORDER BY id DESC"
816             )

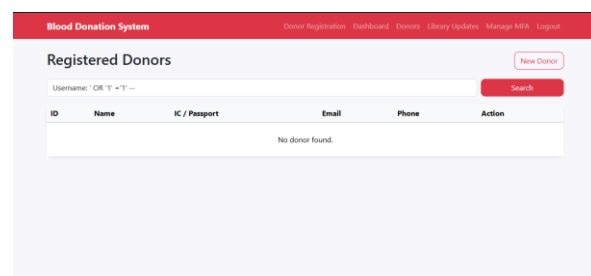
```

Figure 8: Parameterized donor-search query in the secure version



ID	Name	IC / Passport	Email	Phone	Action
1	Almond Zaki	980101-10-1234	almond@sample.com	0123456789	View
2	Siti Aminah	92002-08-5476	si@sample.com	0134567890	View
3	Mickey	134682340	mickey@mickey.com	123456789	View
4	Peter Can-Late	8123772254	plate@pl.com		View

Figure 9(a): Vulnerable search broadened the returned records



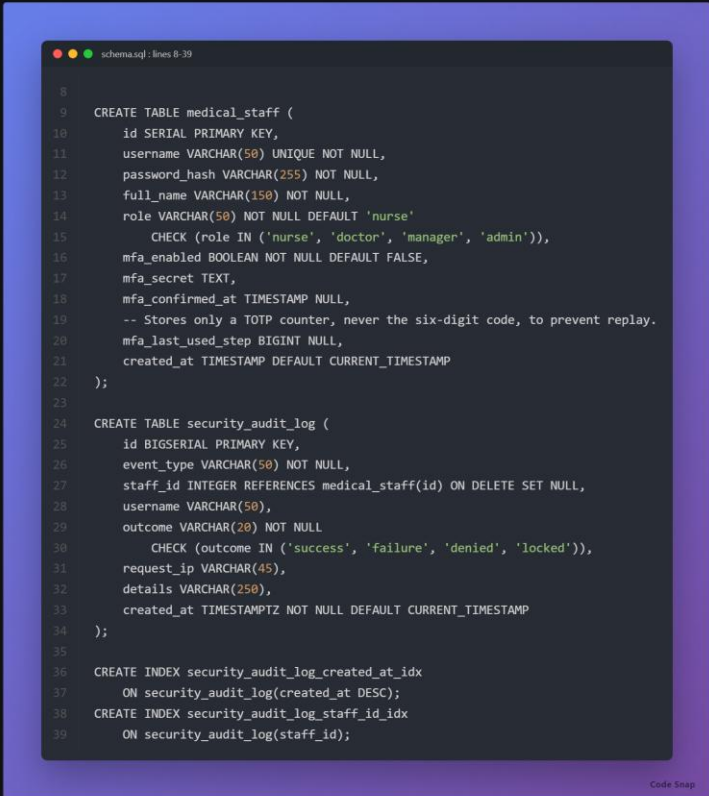
ID	Name	IC / Passport	Email	Phone	Action
No donor found.					

Figure 9(b): Secure search treats the injection-style value as search data

The better evidence is to be found in behaviour, not appearance. The vulnerable build led to records not expected to appear in the search results, as the query logic was altered by the crafted search value. The same "style" of input doesn't expand the search results in the secure build. Later checks by ZAP and sqlmap, done while the application was running, agreed with the above conclusion and provided an additional runtime check for the code-level fix.

3.3 Plaintext Password Storage and Database Migration

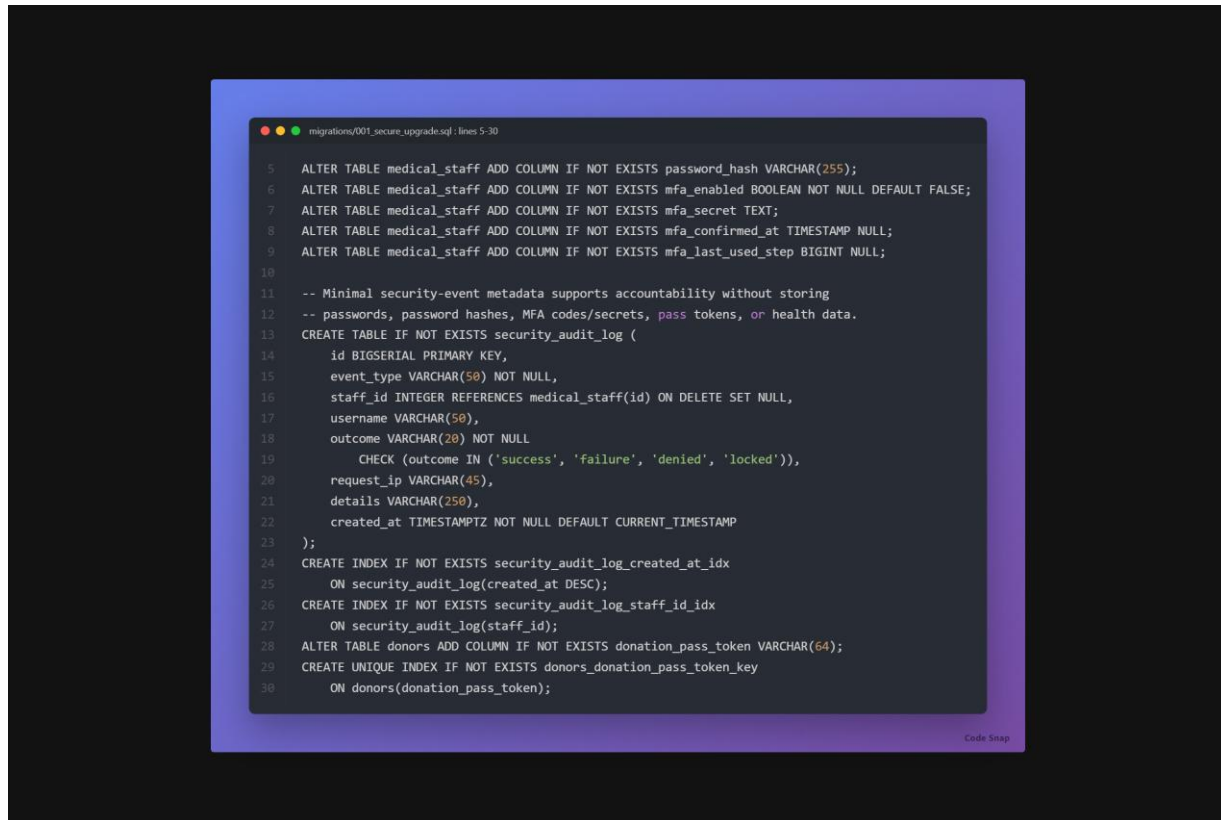
The database schema and the logic for login was modified. The old staff table had a plain text password field, the secure schema has replaced this with password_hash and also added state for MFA. A new table security_audit_log was also added in order to record failed logins, access denials and other security related activities without them being part of the normal donor and donor data.



```
8
9 CREATE TABLE medical_staff (
10     id SERIAL PRIMARY KEY,
11     username VARCHAR(50) UNIQUE NOT NULL,
12     password_hash VARCHAR(255) NOT NULL,
13     full_name VARCHAR(150) NOT NULL,
14     role VARCHAR(50) NOT NULL DEFAULT 'nurse'
15     CHECK (role IN ('nurse', 'doctor', 'manager', 'admin')),
16     mfa_enabled BOOLEAN NOT NULL DEFAULT FALSE,
17     mfa_secret TEXT,
18     mfa_confirmed_at TIMESTAMP NULL,
19     -- Stores only a TOTP counter, never the six-digit code, to prevent replay.
20     mfa_last_used_step BIGINT NULL,
21     created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
22 );
23
24 CREATE TABLE security_audit_log (
25     id BIGSERIAL PRIMARY KEY,
26     event_type VARCHAR(50) NOT NULL,
27     staff_id INTEGER REFERENCES medical_staff(id) ON DELETE SET NULL,
28     username VARCHAR(50),
29     outcome VARCHAR(20) NOT NULL
30     CHECK (outcome IN ('success', 'failure', 'denied', 'locked')),
31     request_ip VARCHAR(45),
32     details VARCHAR(250),
33     created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP
34 );
35
36 CREATE INDEX security_audit_log_created_at_idx
37     ON security_audit_log(created_at DESC);
38 CREATE INDEX security_audit_log_staff_id_idx
39     ON security_audit_log(staff_id);
```

Figure 10: Secure staff schema with password_hash, MFA state and security audit log

A clean reinstall would have been the easiest way to demonstrate the new schema, but it would not say much about maintainability. Rather, a migration script was provided to enable the safe migration of an existing lab database. The migration is replay-safe and includes all the fields and supporting structures needed for the secure application. In practice, the difference is that security enhancements have to have an upgrade path or it's possible to have an existing deployment that's stuck with insecure defaults because it's difficult to rebuild.

A screenshot of a code editor window with a dark background and light blue borders. The editor shows a SQL migration script. The script includes several ALTER TABLE statements to add new columns to the 'medical_staff' table, such as 'password_hash', 'mfa_enabled', 'mfa_secret', 'mfa_confirmed_at', and 'mfa_last_used_step'. It also includes a CREATE TABLE statement for 'security_audit_log' with various fields and constraints, followed by CREATE INDEX statements for 'security_audit_log'. The script ends with ALTER TABLE and CREATE UNIQUE INDEX statements for the 'donors' table. The code is syntax-highlighted, and line numbers are visible on the left side of the editor.

```
migrations/001_secure_upgrade.sql: lines 5-30
5 ALTER TABLE medical_staff ADD COLUMN IF NOT EXISTS password_hash VARCHAR(255);
6 ALTER TABLE medical_staff ADD COLUMN IF NOT EXISTS mfa_enabled BOOLEAN NOT NULL DEFAULT FALSE;
7 ALTER TABLE medical_staff ADD COLUMN IF NOT EXISTS mfa_secret TEXT;
8 ALTER TABLE medical_staff ADD COLUMN IF NOT EXISTS mfa_confirmed_at TIMESTAMP NULL;
9 ALTER TABLE medical_staff ADD COLUMN IF NOT EXISTS mfa_last_used_step BIGINT NULL;
10
11 -- Minimal security-event metadata supports accountability without storing
12 -- passwords, password hashes, MFA codes/secrets, pass tokens, or health data.
13 CREATE TABLE IF NOT EXISTS security_audit_log (
14   id BIGSERIAL PRIMARY KEY,
15   event_type VARCHAR(50) NOT NULL,
16   staff_id INTEGER REFERENCES medical_staff(id) ON DELETE SET NULL,
17   username VARCHAR(50),
18   outcome VARCHAR(20) NOT NULL
19   CHECK (outcome IN ('success', 'failure', 'denied', 'locked')),
20   request_ip VARCHAR(45),
21   details VARCHAR(250),
22   created_at TIMESTAMPTZ NOT NULL DEFAULT CURRENT_TIMESTAMP
23 );
24 CREATE INDEX IF NOT EXISTS security_audit_log_created_at_idx
25   ON security_audit_log(created_at DESC);
26 CREATE INDEX IF NOT EXISTS security_audit_log_staff_id_idx
27   ON security_audit_log(staff_id);
28 ALTER TABLE donors ADD COLUMN IF NOT EXISTS donation_pass_token VARCHAR(64);
29 CREATE UNIQUE INDEX IF NOT EXISTS donors_donation_pass_token_key
30   ON donors(donation_pass_token);
```

Figure 11: Replay-safe database migration for the secure schema

Credential material is now kept out of reports and CSV exports. That includes hashes, and plaintext passwords. A hash is not required on an Operational Report and if it is revealed, there would still be unnecessary authentication risk. By removing those fields, the problem of storing the password is solved, and the problem of too much data is solved as well.

3.4 RBAC and Operational Report Data Minimisation

In the old Operational Report there was not much more than a generic 'staff is logged in' check, and thus ordinary clinical accounts could bring in to a high valued collection of information. The secure build includes additional reusable `role_required`, `login_required` controls and denied

access is recorded in the audit log. OWASP A01:2025 defines broken access control to be a condition where a user is granted access to files or is given privileges that the user was not originally meant to have, which is very similar to the original behaviour (OWASP Foundation, 2025b).

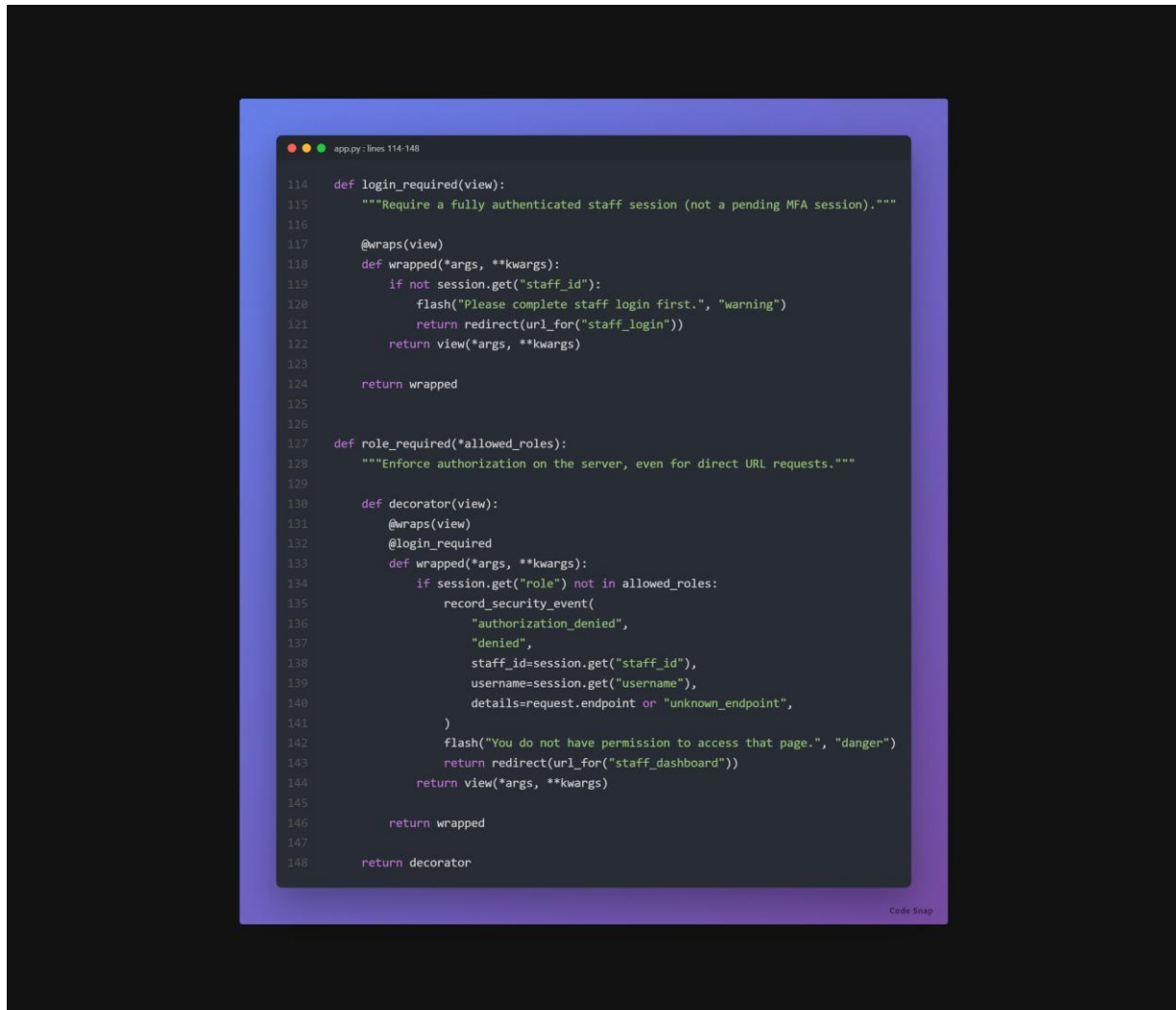


Figure 12: Server-side RBAC helpers and denied-access audit behaviour

The Operational Report is now only available to server-side management and only returns a subset of useful operationally relevant fields in database queries. Only password material is not included, but MFA metadata is. The navigation is also tidier, as the users who don't have the role will not see the report option, but this is just a usability tweak – the real control remains the route-level authorization check.

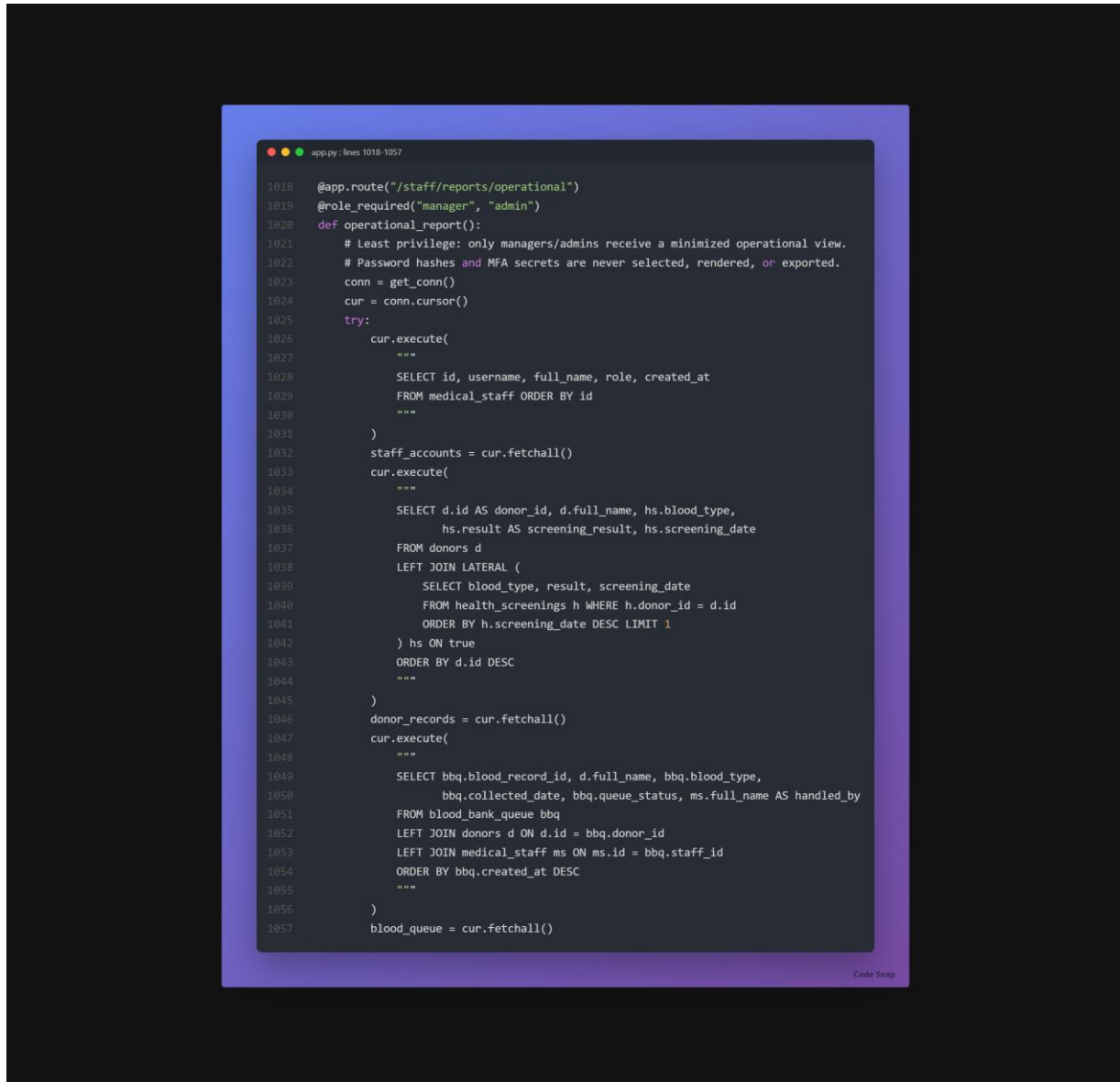


Figure 13: Manager/admin report authorization and data-minimised query

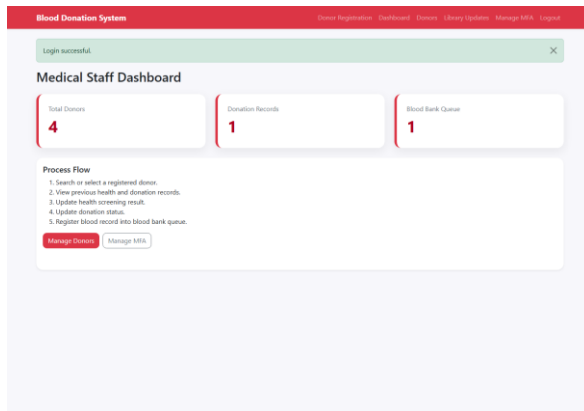


Figure 14(a): Nurse dashboard without privileged report access

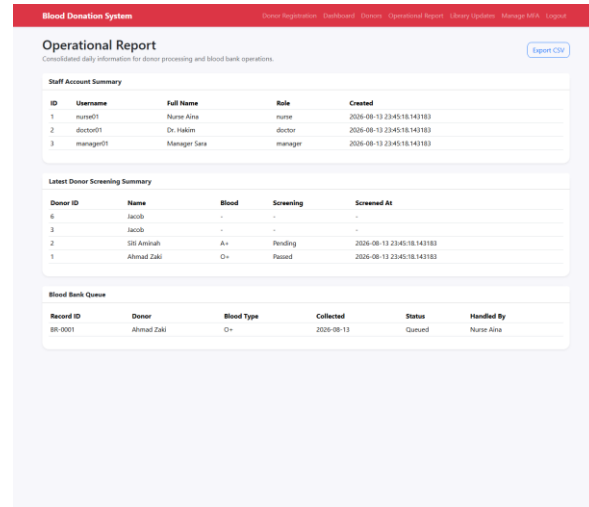
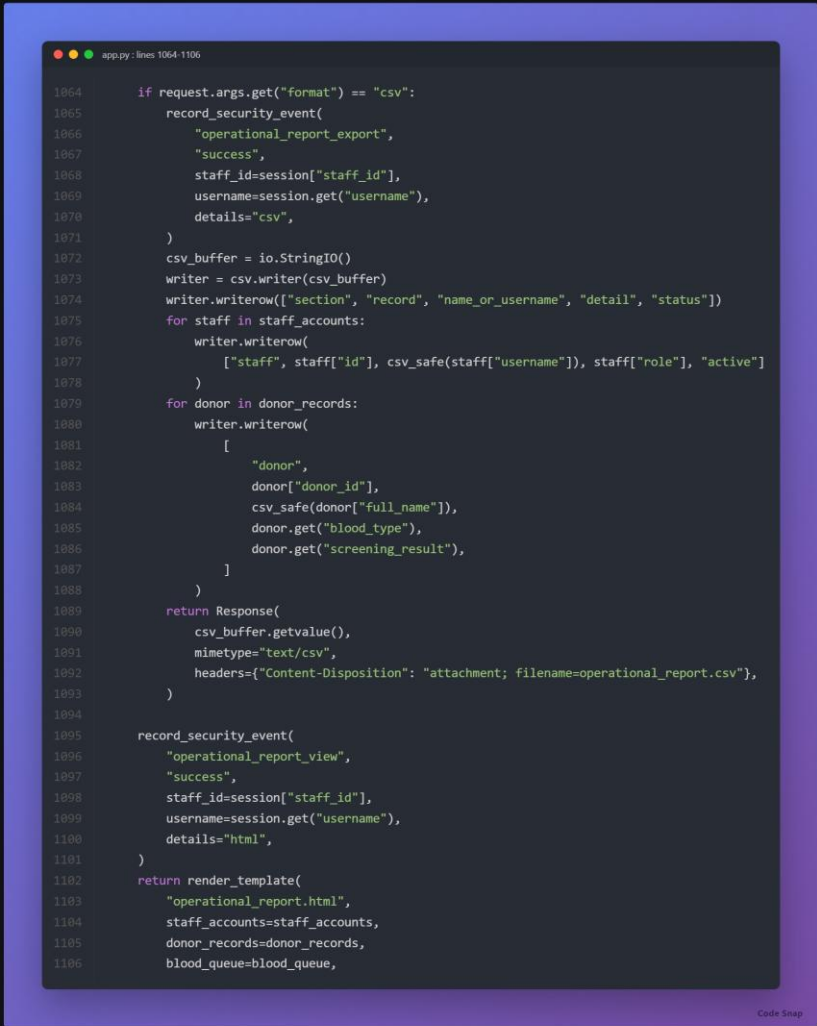


Figure 14(b): Operational report available to an authorized manager

The export to CSV is treated as a sensitive operation. The disclosure is repeated, bulk export is magnified, the required fields are exported, spreadsheet-formula prefixes are stripped out and an audit event is created. The outcome is that there's less surface area to export, which means there's lower risk that a downloaded CSV file will cause a second issue when opened in a spreadsheet program.



```

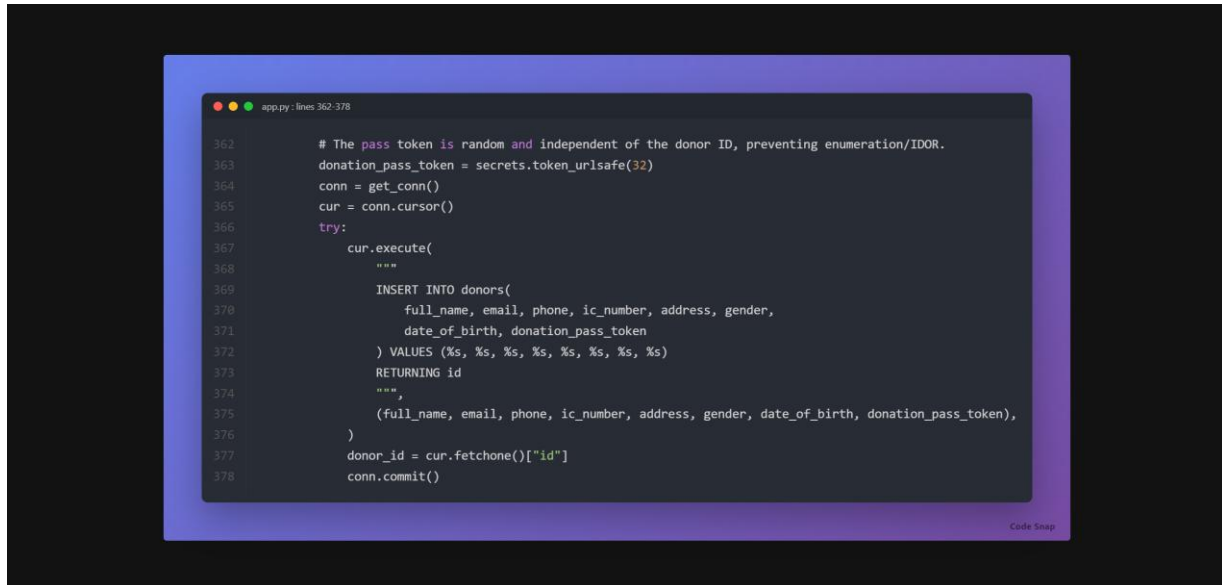
1064     if request.args.get("format") == "csv":
1065         record_security_event(
1066             "operational_report_export",
1067             "success",
1068             staff_id=session["staff_id"],
1069             username=session.get("username"),
1070             details="csv",
1071         )
1072         csv_buffer = io.StringIO()
1073         writer = csv.writer(csv_buffer)
1074         writer.writerow(["section", "record", "name_or_username", "detail", "status"])
1075         for staff in staff_accounts:
1076             writer.writerow(
1077                 ["staff", staff["id"], csv_safe(staff["username"]), staff["role"], "active"]
1078             )
1079         for donor in donor_records:
1080             writer.writerow(
1081                 [
1082                     "donor",
1083                     donor["donor_id"],
1084                     csv_safe(donor["full_name"]),
1085                     donor.get("blood_type"),
1086                     donor.get("screening_result"),
1087                 ]
1088             )
1089         return Response(
1090             csv_buffer.getvalue(),
1091             mimetype="text/csv",
1092             headers={"Content-Disposition": "attachment; filename=operational_report.csv"},
1093         )
1094
1095     record_security_event(
1096         "operational_report_view",
1097         "success",
1098         staff_id=session["staff_id"],
1099         username=session.get("username"),
1100         details="html",
1101     )
1102     return render_template(
1103         "operational_report.html",
1104         staff_accounts=staff_accounts,
1105         donor_records=donor_records,
1106         blood_queue=blood_queue,

```

Figure 15: Restricted CSV export with audit logging and formula neutralisation

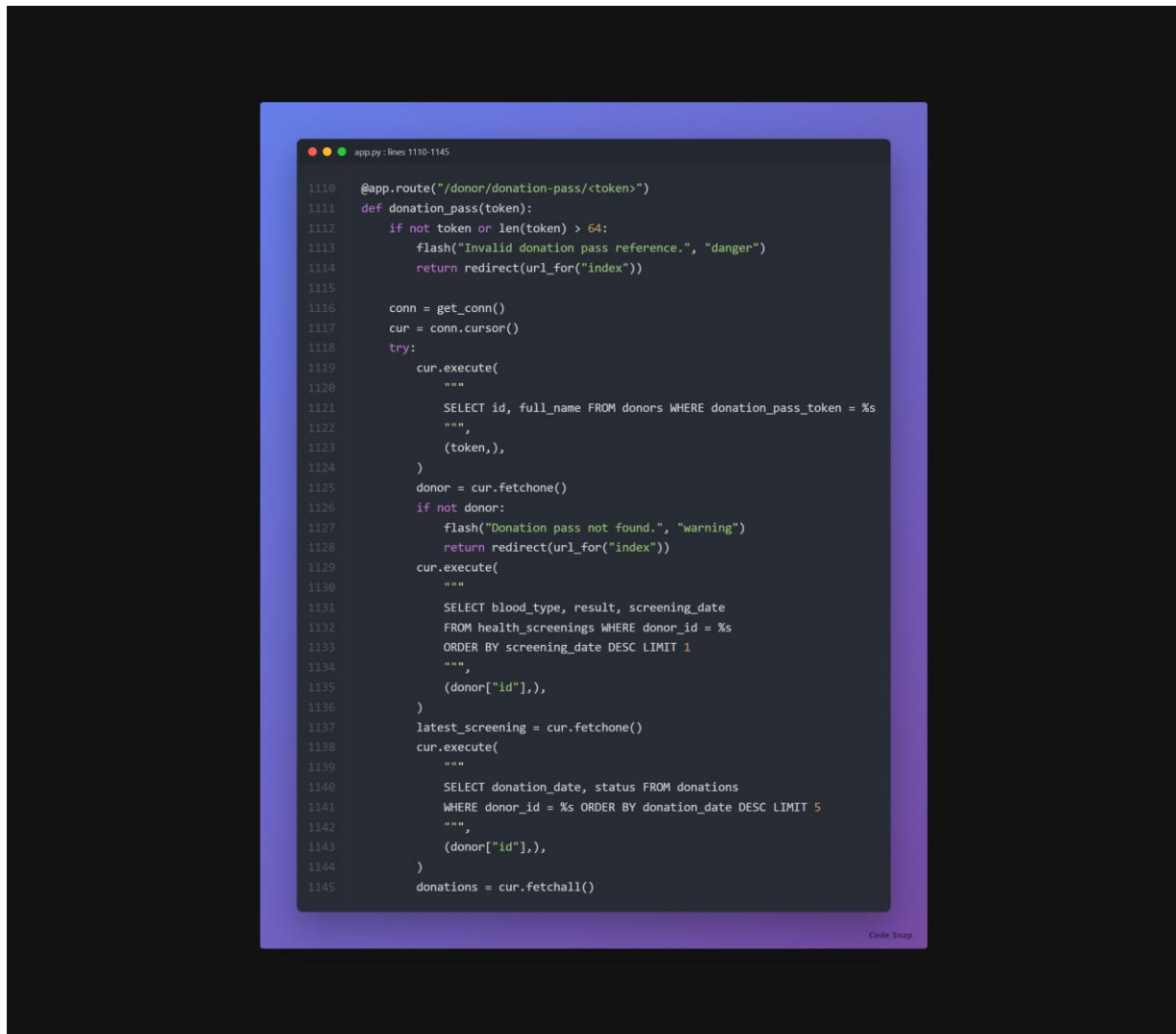
3.5 Predictable Donation Pass / Insecure Direct Object Reference

The old donation-pass value was a straight calculation from donor_id, and provided a clue to many others. Instead of generating a sequential ID to serve as proof of access, the secure version generates a high entropy token using Python secrets module, stores it on the server and resolves the public request using the token. Even though OWASP does not advocate that you implement object-level authorization, they do agree that unpredictable identifiers are a good measure of defence in depth when you cannot avoid having public references (OWASP Foundation, n.d.-d).



```
362 # The pass token is random and independent of the donor ID, preventing enumeration/IDOR.
363 donation_pass_token = secrets.token_urlsafe(32)
364 conn = get_conn()
365 cur = conn.cursor()
366 try:
367     cur.execute(
368         """
369         INSERT INTO donors(
370             full_name, email, phone, ic_number, address, gender,
371             date_of_birth, donation_pass_token
372         ) VALUES (%s, %s, %s, %s, %s, %s, %s, %s)
373         RETURNING id
374         """
375         , (full_name, email, phone, ic_number, address, gender, date_of_birth, donation_pass_token),
376     )
377     donor_id = cur.fetchone()["id"]
378     conn.commit()
```

Figure 16: Cryptographically random donation-pass token generation and storage



```
1110 @app.route("/donor/donation-pass/<token>")
1111 def donation_pass(token):
1112     if not token or len(token) > 64:
1113         flash("Invalid donation pass reference.", "danger")
1114         return redirect(url_for("index"))
1115
1116     conn = get_conn()
1117     cur = conn.cursor()
1118     try:
1119         cur.execute(
1120             """
1121             SELECT id, full_name FROM donors WHERE donation_pass_token = %s
1122             """
1123             , (token,)
1124         )
1125         donor = cur.fetchone()
1126         if not donor:
1127             flash("Donation pass not found.", "warning")
1128             return redirect(url_for("index"))
1129         cur.execute(
1130             """
1131             SELECT blood_type, result, screening_date
1132             FROM health_screenings WHERE donor_id = %s
1133             ORDER BY screening_date DESC LIMIT 1
1134             """
1135             , (donor["id"],)
1136         )
1137         latest_screening = cur.fetchone()
1138         cur.execute(
1139             """
1140             SELECT donation_date, status FROM donations
1141             WHERE donor_id = %s ORDER BY donation_date DESC LIMIT 5
1142             """
1143             , (donor["id"],)
1144         )
1145         donations = cur.fetchall()
```

Figure 17: Server-side verification of the secure donation-pass token

This eliminates the old BDP-`{id}` pattern, so that adding or guessing a donor number will not yield another valid pass. Public information was also restricted to the information required for the pass. The only problem currently is that there is no end-user expiry or rotation workflow for the bearer token. That is what I've left on the table in section 5 but not presented as a complete lifecycle solution.

3.6 Stored Cross-Site Scripting

Stored XSS was possible due to the ability to store fields with script capable text, and because of the original application's deliberate omission of Jinja escaping via a custom Markup filter. The shortcut is not included in the secure version. Untrusted text remains text rather than being rendered as executable markup due to left to Jinja's normal auto-escaping. This is based on OWASP recommendations to use framework output encoding and safe rendering contexts, instead of manually declaring untrusted content safe (OWASP Foundation, n.d.-e).

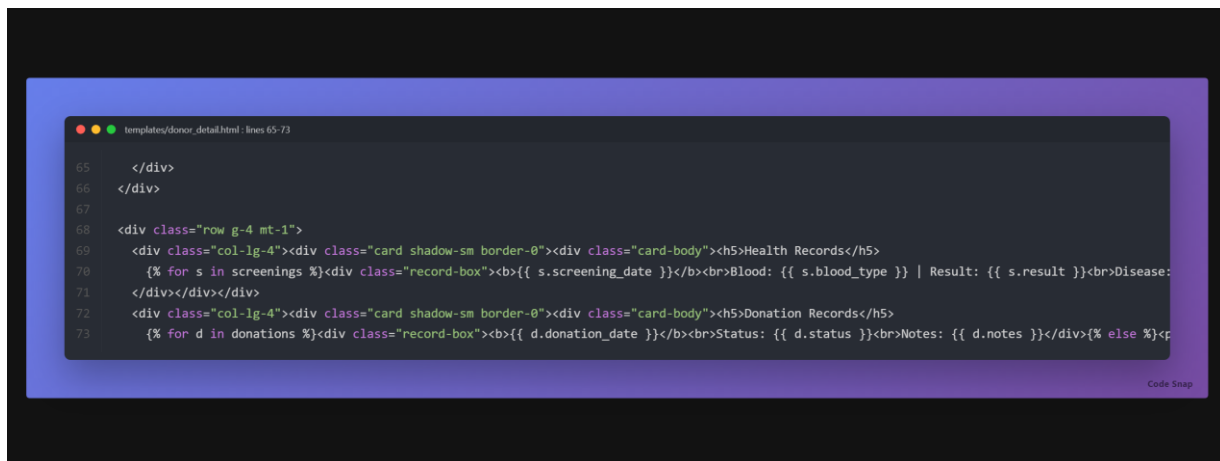


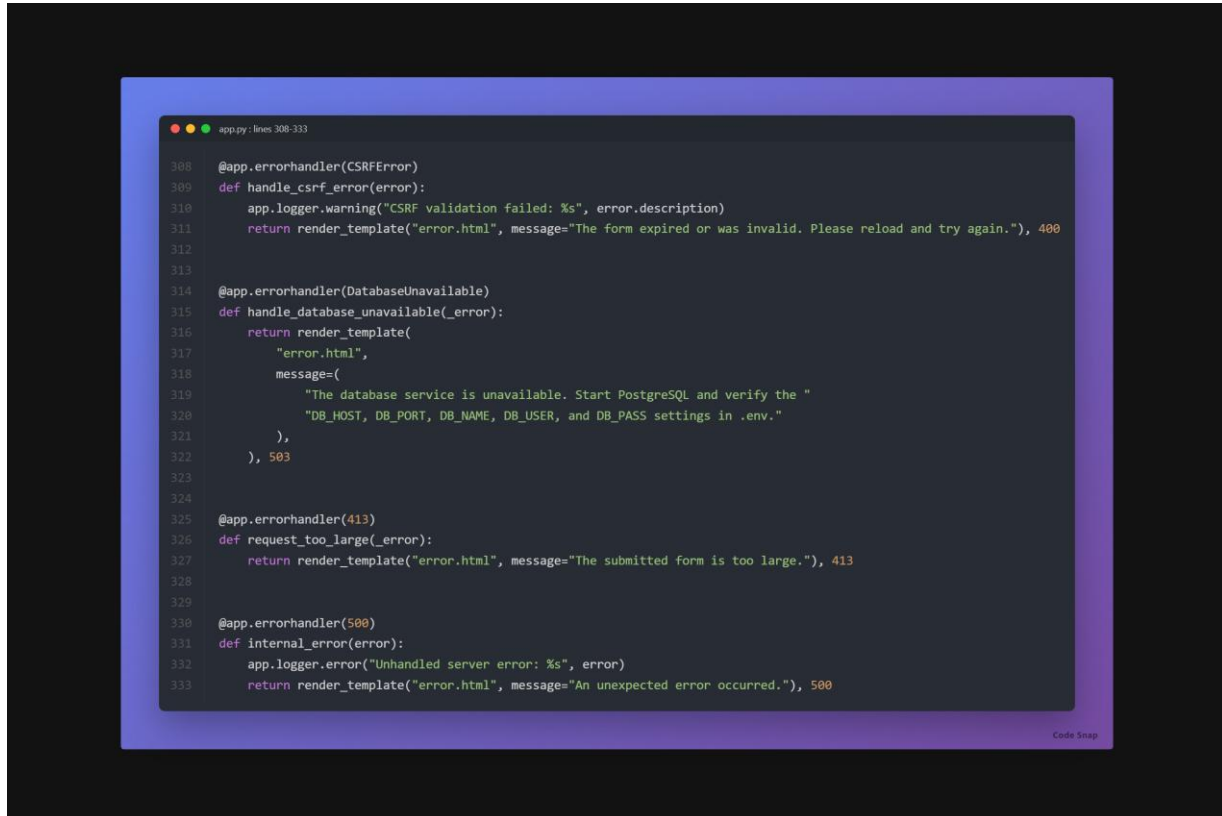
Figure 18: Jinja auto-escaping used for stored health-screening content

They even included length checks and input validation – though these are supporting controls, not the primary XSS defense. The security requirement is simple: If, for example, a script like `<script>alert(1)</script>` is stored, the browser should output the characters, rather than run the script. It's helpful to have that expectation captured by a regression test because a subsequent template edit may re-introduce the same weakness silently.

3.7 Cross-Site Request Forgery Protection

CSRF protection is turned on at the global level with Flask-WTF, and errors and missing tokens are diverted to a secure error path. Flask-WTF lists CSRFProtect as the application-wide solution

for protected views, and requires a `csrf_token` value to be passed in by forms (Flask-WTF, 2026). This build impacts the staff login, donor registration, clinical updates and MFA management, as opposed to just one or two forms.



```
app.py : lines 308-333

308 @app.errorhandler(CSRFError)
309 def handle_csrf_error(error):
310     app.logger.warning("CSRF validation failed: %s", error.description)
311     return render_template("error.html", message="The form expired or was invalid. Please reload and try again."), 400
312
313
314 @app.errorhandler(DatabaseUnavailable)
315 def handle_database_unavailable(_error):
316     return render_template(
317         "error.html",
318         message=(
319             "The database service is unavailable. Start PostgreSQL and verify the "
320             "DB_HOST, DB_PORT, DB_NAME, DB_USER, and DB_PASS settings in .env."
321         ),
322     ), 503
323
324
325 @app.errorhandler(413)
326 def request_too_large(_error):
327     return render_template("error.html", message="The submitted form is too large."), 413
328
329
330 @app.errorhandler(500)
331 def internal_error(error):
332     app.logger.error("Unhandled server error: %s", error)
333     return render_template("error.html", message="An unexpected error occurred."), 500
```

Figure 19: Global CSRF protection and safe CSRF/error handlers

```

26 <form method="post" action="{{ url_for('update_screening', donor_id=donor.id) }}">
27 <input type="hidden" name="csrf_token" value="{{ csrf_token() }}">
28 <div class="row g-3">
29 <div class="col-md-4"><label class="form-label">Blood Type</label><select name="blood_type" class="form-select"><option>A</option><option>
30 <div class="col-md-4"><label class="form-label">Weight (kg)</label><input name="weight" class="form-control" type="number" step="0.1"></div>
31 <div class="col-md-4"><label class="form-label">Height (cm)</label><input name="height" class="form-control" type="number" step="0.1"></div>
32 <div class="col-md-6"><label class="form-label">Disease / Condition</label><input name="disease" class="form-control" maxlength="300" plac
33 <div class="col-md-6"><label class="form-label">Result</label><select name="result" class="form-select"><option>Passed</option><option>Fai
34 <div class="col-12"><label class="form-label">Remarks</label><textarea name="remarks" class="form-control" maxlength="500" rows="2"></text
35 </div>
36 <button class="btn btn-danger mt-3">Save Screening</button>
37 </form>
38 </div></div>
39
40 <div class="card shadow-sm border-0 mb-4"><div class="card-body">
41 <h5>Update Donation Status</h5>
42 <form method="post" action="{{ url_for('update_donation', donor_id=donor.id) }}">
43 <input type="hidden" name="csrf_token" value="{{ csrf_token() }}">
44 <div class="row g-3">
45 <div class="col-md-5"><label class="form-label">Donation Status</label><select name="donation_status" class="form-select"><option>Pending
46 <div class="col-md-7"><label class="form-label">Notes</label><input name="notes" class="form-control" maxlength="500"></div>
47 </div>
48 <button class="btn btn-danger mt-3">Update Status</button>
49 </form>
50 </div></div>
51
52 <div class="card shadow-sm border-0"><div class="card-body">
53 <h5>Register Blood Record into Blood Bank Queue</h5>
54 <form method="post" action="{{ url_for('create_blood_record', donor_id=donor.id) }}">
55 <input type="hidden" name="csrf_token" value="{{ csrf_token() }}">
56 <div class="row g-3">

```

Figure 20: CSRF tokens embedded in state-changing staff forms

Figure 21(a): Secure donor registration form

Figure 21(b): Missing-CSRF request rejected with HTTP 400

Just coming across a hidden CSRF field in the HTML is insufficient to show that the control is working. The better test is to delete it and send the same request to change the state again. For that check it was done using ZAP, and the server denied the changed request. While highly convenient, SameSite cookie settings are not used in place of CSRF tokens, rather as a second layer of browser restriction.

3.8 Session Security and Security Response Headers

Configured the session at the same time. The old hardcoded Flask secret has been taken away, HttpOnly is no longer disabled, SameSite restrictions are applied and the Secure attribute can now be enabled when deployed over HTTPS. Secure, HttpOnly and SameSite are emphasized by Flask's security guidance and the OWASP session guidance as they provide different aspects of session exposure protection: transport, access via JavaScript, and cross-site submission behaviour (Pallets Projects, 2026b; OWASP Foundation, n.d.-f).

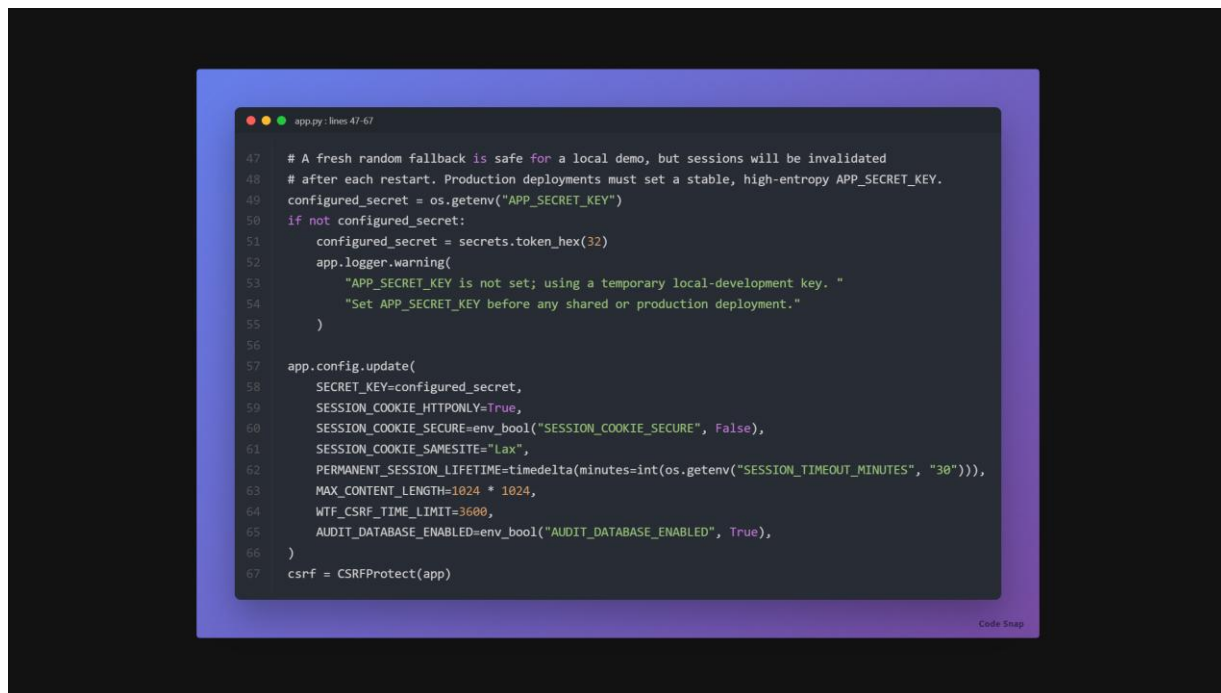


Figure 22: Environment-based Flask secret and hardened session-cookie configuration

The application also sends X-Frame-Options, X-Content-Type-Options and Referrer-Policy response headers. The settings directly address the hardening gap that Nikto indicated in the old build, and limit the leakage of referrer information, the sniffing of MIME types and the leak of unnecessary referrer information. It's not so complicated as to be difficult for the local lab, but it does help to clarify the expected production posture.

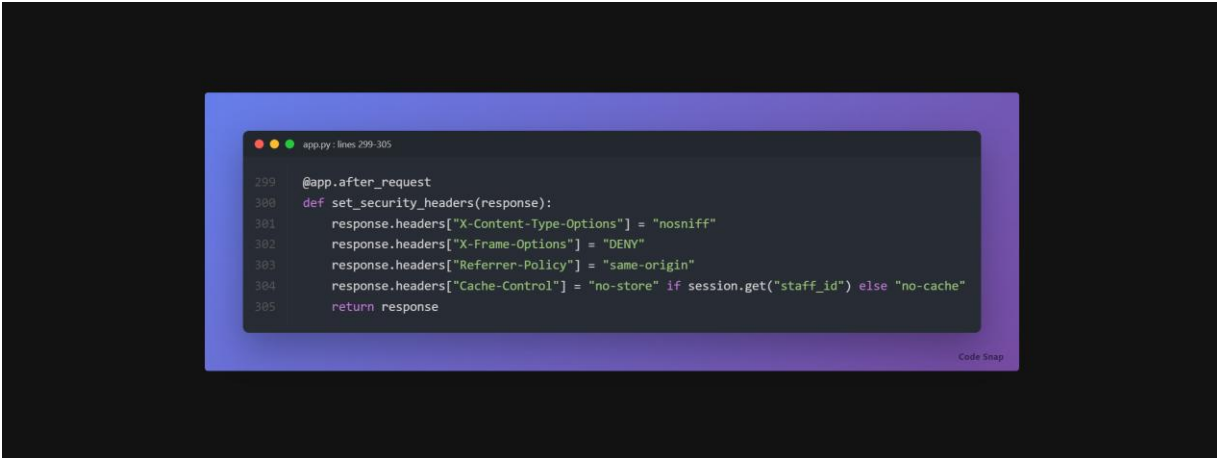


Figure 23: Security headers applied to application responses



Figure 24: ZAP evidence of the hardened session cookie in the secure application

3.9 Safe Error Handling, Logging and Additional Hardening

In the previous version, it'd return raw exception text, even making SQL on the fly in debug mode. For the secure build, the error handling is done in a centralised manner. The users get an old fashioned reply, and valuable diagnostic information is left on the server. This aligns with the approach recommended by OWASP to differentiate between user-facing error messages and controlled internal diagnostics (OWASP Foundation, n.d.-g). The change will allow the system to look cleaner, as well as eliminate valuable reconnaissance information from malformed requests and database errors.

Security events are now recorded at application level and not solely in the web-server's log files. A parameterized audit helper captures authentication results, authorization failures and sensitive report/export activity. That context is important in an investigation: Infrastructure logs can

inform the investigator that a request happened, but cannot tell them what security decision was taken by the application, or why (OWASP Foundation, n.d.-h).



Figure 25: Parameterized security audit logging helper

As testing continued a few smaller controls were added. Login-rate helpers minimize repeated login attempts. Screening measurements are checked by finite number, which means that some values like NaN and infinity are not passed by numeric comparison. A subprocess argument list with shell=False and timeout is used by the dependency checker. Last but not least, the test suite now includes support for throttling and replay/audit behaviour of TOTP. These were not all part of the first remediation pass, but were the result of scanner and test feedback which is just the kind of iteration a secure development pipeline should yield.

```

196 def update_screensize(id):
197     try:
198         blood_type = clean_text(request.form.get('blood_type'), 1, require=True)
199         disease = clean_text(request.form.get('disease'), 100)
200         result = clean_text(request.form.get('result'), 30, require=True)
201         remarks = clean_text(request.form.get('remarks'), 100)
202         weight = para_screensize_measurement(
203             request.form.get('weight'), height=36, sex='m')
204     except:
205         height = para_screensize_measurement(
206             request.form.get('height'), weight=386, sex='m')
207     if blood_type is not ALLNone and 0.9993 is result or is in ("Pass", "Fail", "Pending"):
208         raise ValueError("Invalid screen size selection.")
209     except ValueError as error:
210         flash(error, 'danger')
211     return redirect(url_for('show_detail', disease_id=id))

```

Figure 26(b): Screening measurement validation

```

1500
1501 #app.routes["/loff/library-updates"]
1502
1503 @app.route("/library-updates")
1504 def library_updates():
1505     try:
1506         result = subprocess.run(
1507             ["sysctladm", "-w", "lib", "list", "--noheader", "--format=columns"],
1508             stdout=subprocess.PIPE,
1509             text=True,
1510             timeout=5,
1511             check=True,
1512         )
1513     except subprocess.CalledProcessError as e:
1514         if result.returncode == 0:
1515             output = "result.stdout or 'All installed libraries are up to date.'{}".format(
1516                 result.stdout
1517             )
1518         else:
1519             app.logger.warning("lib updated check failed with exit code %s", result.returncode)
1520             output = "The update check could not be completed, try the command from a local terminal."
1521     except subprocess.SubprocessError as e:
1522         app.logger.exception("Library update check failed")
1523         output = "The update check could not be completed, try the command from a local terminal."
1524     return render_template("library_updates.html", output=output)
1525
1526

```

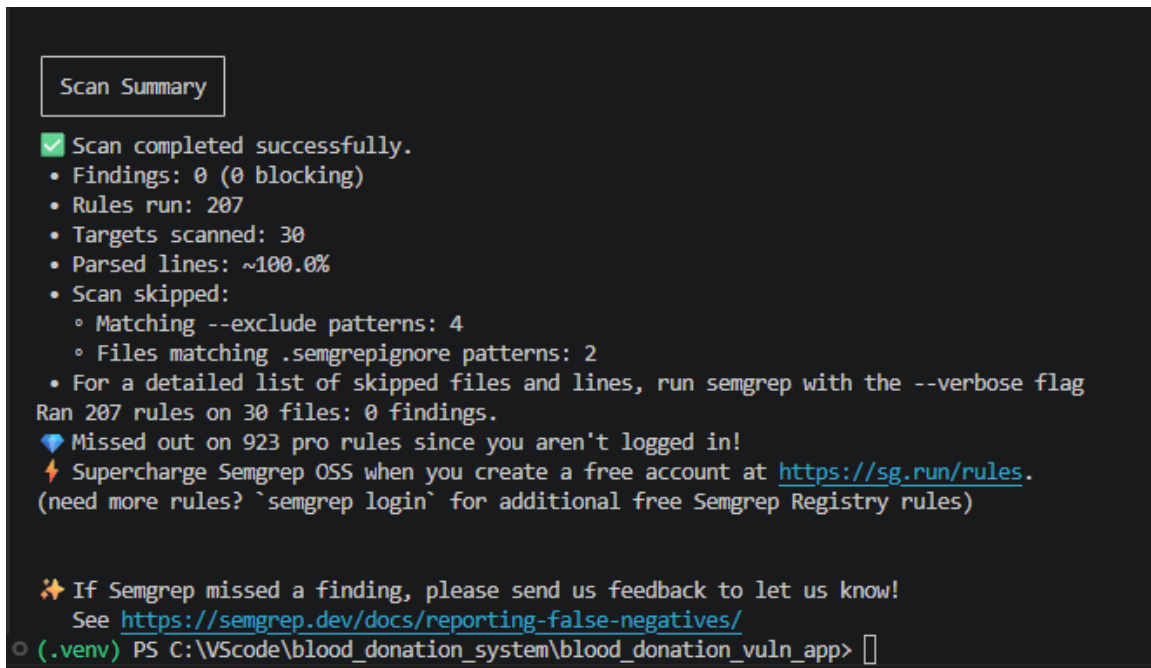
Figure 27(b): Controlled dependency update check

A new application was retested with the same combination of tools suggested in Part 1. Sengrep and Bandit: Identified source level patterns, ZAP: Tested live HTTP traffic, sqlmap: Played through 2 known injection points, pip-audit: Checked published dependency advisories, pytest: Exercised security regression cases, and Nikto: Tested server/header exposure. I didn't score number of findings. There are different tools with different rulesets/featuresets and the question

is, does the outcome of this tool align with the implemented control? A clean scan is a good thing, it does not mean there is no vulnerability.

4.1 Semgrep

The focused Semgrep scan analyzed 30 project targets, 207 rules and did not report any results. That's an improvement of 30% over the insecure baseline, but it shouldn't be compared with the two numbers as a single numerical security rating as they were not the same scanned target sets. The end effect is a simpler one: the secure project doesn't show up any of these patterns or blocking findings in this scan. Since each result is associated with a file and a rule (Semgrep, n.d.), repeated local checks are well suited to be done with Semgrep.



```
Scan Summary
✅ Scan completed successfully.
  • Findings: 0 (0 blocking)
  • Rules run: 207
  • Targets scanned: 30
  • Parsed lines: ~100.0%
  • Scan skipped:
    ◦ Matching --exclude patterns: 4
    ◦ Files matching .semgrepignore patterns: 2
  • For a detailed list of skipped files and lines, run semgrep with the --verbose flag
Ran 207 rules on 30 files: 0 findings.
💎 Missed out on 923 pro rules since you aren't logged in!
⚡ Supercharge Semgrep OSS when you create a free account at https://sg.run/rules.
(need more rules? `semgrep login` for additional free Semgrep Registry rules)

🌟 If Semgrep missed a finding, please send us feedback to let us know!
See https://semgrep.dev/docs/reporting-false-negatives/
○ (.venv) PS C:\VScode\blood_donation_system\blood_donation_vuln_app> 
```

Figure 28: Semgrep secure-project scan reporting zero findings

4.2 Bandit

Bandit was executed without virtual environments, evidence folders and tests against the application code. It reported four Low and no Medium & High findings. All four Low results are part of the controlled dependency-update feature of the subprocess module of Python. Manual review revealed that there was no shell=True, no user-controlled command content and a timeout. The warnings were not ignored to get a better-looking screenshot but were allowed to stand and recorded as a low-risk identified review. It's a much better use of a static analyzer than assuming every warning is a vulnerability or that it's not and pretending it's not. (PyCQA, 2026).

```

Code scanned:
  Total lines of code: 885
  Total lines skipped (#nosec): 0

Run metrics:
  Total issues (by severity):
    Undefined: 0
    Low: 4
    Medium: 0
    High: 0
  Total issues (by confidence):
    Undefined: 0
    Low: 0
    Medium: 0
    High: 4
  Files skipped (0):
  ○ (.venv) PS C:\Vscode\blood_donation_system\blood_donation_vuln_app>

```

Figure 29: Bandit result for the fixed application - four Low, zero Medium/High findings

4.3 OWASP ZAP

The browser proxy was used as ZAP in order to compare the old and new HTTP behaviour straight. The most obvious outcome of this was when staff login - this was to be a value injected into the database, which was previously not being authenticated, and the secure build rejected it. This was the case also when searching for donors, as the value used was no longer used to expand returned records. In addition, ZAP gave evidence to the CSRF enforcement, session-cookie attributes, and redirect for MFA. Its passive analysis is useful for observing proxied traffic without altering it, while request editing and resend make it possible to change one security condition at a time when a controlled negative test is needed (ZAP by Checkmarx, 2026).

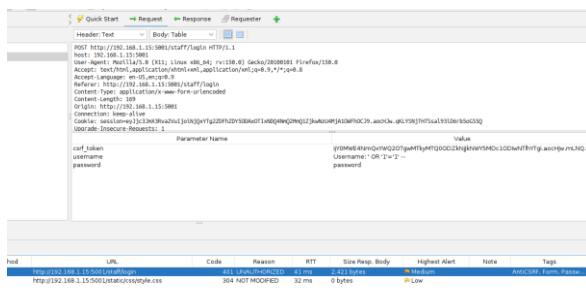


Figure 30(a): ZAP - fixed login rejects SQL injection



Figure 30(b): ZAP - fixed donor search handles injection-style input safely

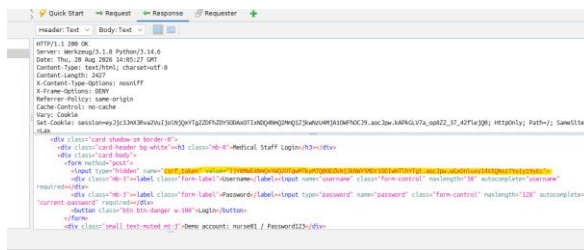


Figure 31(a): ZAP confirms CSRF token in the secure form

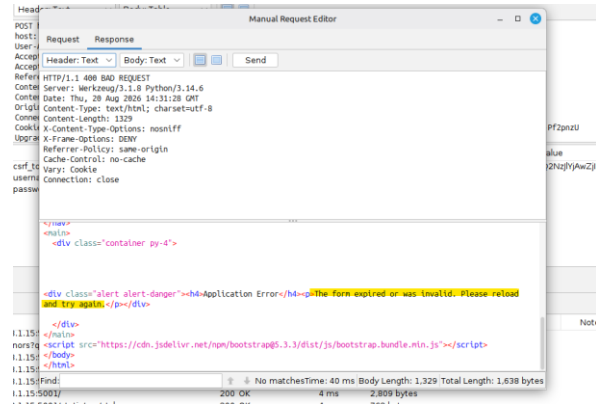


Figure 31(b): ZAP resend without CSRF token is rejected



Figure 32(a): Password success redirects to MFA verification

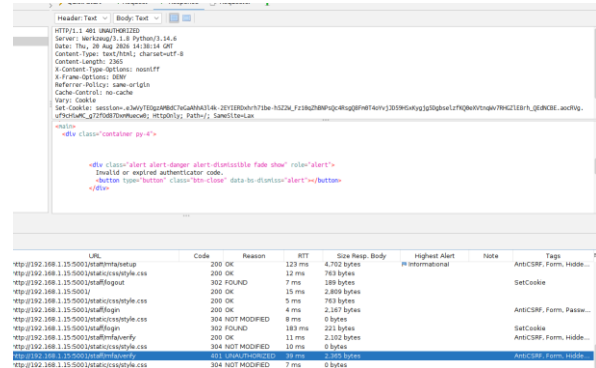


Figure 32(b): Invalid MFA code is rejected

4.4 sqlmap

sqlmap was kept for a narrow purpose: rechecking the exact SQL injection risk. It can target GET and POST parameters and recognise DBMS-specific injection behaviour, which made it a good independent check of the two endpoints (sqlmap project, 2026). The vulnerable build had been confirmed injectable at both staff login and donor search. Against the secure build, the same class of testing no longer confirmed injection in the login flow and did not detect it in the donor-search parameter. That result is consistent with the source change from interpolated SQL strings to psycopg2 parameter binding.

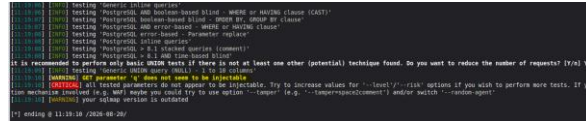


Figure 33(b): sqlmap does not detect injection in the improved donor search

The first dependency scan of the improved environment still found two published advisories, affecting `python-dotenv` 1.0.1 and `pytest` 8.3.2. Both packages were upgraded to the fix versions identified by `pip-audit`, `requirements.txt` was updated, and the tests were run again before a second audit. The follow-up scan reported no known vulnerabilities. This was a useful reminder that secure coding is not only about code written for the application; package maintenance is part of the same release process. `pip-audit` can only report known package advisories, so its clean result complements rather than replaces source and runtime testing (Python Packaging Authority, 2026).

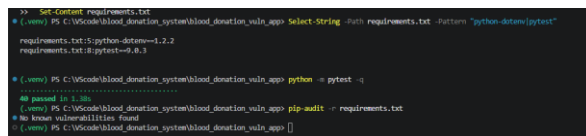


Figure 34(b): Dependency update completed and pip-audit reports no known vulnerabilities

pytest provides the repeatable acceptance layer for the revised system. The suite includes authentication, MFA, rate controls, authorization, CSRF, XSS, donation tokens, sessions and error handling. However, the dependency updates and then the full run passed, with 40 passing tests, and the security-focused one also passed. Also, a Python compile check was performed on the application and supporting scripts. Useful for this assignment are screenshots, but when later the code changes, the same expected security behaviours are checked again automatically: regression tests (pytest developers, 2026).

```
>> Set-Content requirements.txt
(.venv) PS C:\VScode\blood_donation_system\blood_donation_vuln_app> Select-String -Path requirements.txt -Pattern "python-dotenv|pytest"

requirements.txt:5:python-dotenv==1.2.2
requirements.txt:8:pytest==9.0.3

(.venv) PS C:\VScode\blood_donation_system\blood_donation_vuln_app> python -m pytest -q
.....
40 passed in 1.38s
(.venv) PS C:\VScode\blood_donation_system\blood_donation_vuln_app> pip-audit -r requirements.txt
No known vulnerabilities found
(.venv) PS C:\VScode\blood_donation_system\blood_donation_vuln_app> 
```

Figure 35: Full regression suite after dependency updates - 40 passed and no known package vulnerabilities

```
(.venv) PS C:\VScode\blood_donation_system\blood_donation_vuln_app> python -m pytest --q tests\test_security.py
.....
40 passed in 1.26s
```

Figure 36(a): Security-specific pytest run passed

```
PS C:\VScode\blood_donation_system\blood_donation_vuln_app> python -m py_compile app.py scripts\migrate_secure.py scripts\check_updates.py tests\test_s
ecurity.py
(.venv) PS C:\VScode\blood_donation_system\blood_donation_vuln_app> 
```

Figure 36(b): Python compile verification completed successfully

4.7 Nikto

Nikto was used as an aid for deployment and response-header hardening. The vulnerable build featured the Werkzeug/Python banner, no X-Frame-Options and an interesting /console path. The secure scan detects the development-server banner - as it should be in this lab - and also detects Referrer-Policy: same-origin, X-Content-Type-Options: nosniff and X-Frame-Options: DENY. The above headers are just markers of security hardening, not an indication of overall security. A production deployment would still switch out Flask's development server for a production WSGI stack and HTTPS.

```
niks@Hemanth-virtual-machine:~$ nikto -h http://127.0.0.1:5000
- Nikto v2.1.5
-----
+ Target IP: 127.0.0.1
+ Target Hostname: localhost
+ Target Port: 5000
+ Start Time: 2026-08-20 11:45:27 (GMT-4)
-----
+ Server: Werkzeug/3.0.3 Python/3.12.3
+ The anti-clickjacking X-Frame-Options header is not present.
+ No CGI Directories found (use '-C all' to force check all possible dirs)
+ Allowed HTTP Methods: OPTIONS, HEAD, GET
+ OSVDB-3092: /console: This might be interesting...
+ 6544 items checked: 0 error(s) and 3 item(s) reported on remote host
+ End Time: 2026-08-20 11:45:42 (GMT-4) (15 seconds)
-----
+ 1 host(s) tested
niks@Hemanth-virtual-machine:~$ 
```

Figure 37(a): Nikto baseline - missing anti-clickjacking header

```
niks@Hemanth-virtual-machine:~$ nikto -h http://192.168.1.15:5001
- Nikto v2.1.5
-----
+ Target IP: 192.168.1.15
+ Target Hostname: 192.168.1.15
+ Target Port: 5001
+ Start Time: 2026-08-20 11:52:51 (GMT-4)
-----
+ Server: Werkzeug/3.1.8 Python/3.14.6
+ Uncommon header 'referrer-policy' found, with contents: same-origin
+ Uncommon header 'x-content-type-options' found, with contents: nosniff
+ Uncommon header 'x-frame-options' found, with contents: DENY
+ No CGI Directories found (use '-C all' to force check all possible dirs)
+ Allowed HTTP Methods: HEAD, OPTIONS, GET
+ 6544 items checked: 0 error(s) and 4 item(s) reported on remote host
+ End Time: 2026-08-20 11:53:15 (GMT-4) (24 seconds)
-----
+ 1 host(s) tested
```

Figure 37(b): Nikto secure scan - security headers present

4.8 Consolidated Secure Testing Result

Tool	Baseline observation	Post-remediation result	Assessment
Semgrep	Vulnerable baseline produced	Focused fixed-project scan: 0 findings	Pass with scope

	numerous findings		caveat
Bandit	Old code included high/low Python security warnings	4 Low; 0 Medium; 0 High after excluding tests	Pass; reviewed subprocess warnings
OWASP ZAP	SQLi behaviour, weak cookie/error/CSRF evidence	SQLi rejected, CSRF enforced, MFA redirect and hardened cookie observed	Pass
sqlmap	Login and donor-search parameters confirmed injectable	No injection confirmed/detected on the improved endpoints	Pass
pip-audit	Old requirements contained multiple published advisories	2 advisories initially; packages updated; no known vulnerabilities afterward	Pass after remediation
pytest	Not applicable to vulnerable baseline as security regression evidence	40 tests passed after updates; security-specific tests passed	Pass
Nikto	Missing X-Frame-Options and server information disclosure	X-Frame-Options DENY, nosniff and same-origin observed	Improved; dev-server banner remains

Table 4: Consolidated secure testing results

It is important to remember that none of these results is decisive. The findings are static, keeping it in sync with the source changes; the HTTP tests match the expected runtime behaviour; the dependency audit ensures that known package advisories were dealt with; and the controls are repeatable using pytest. The best evidence is the before/after negative retest: queries that previously used authentication SQL or donor search SQL that were changed no longer do so, and missing-CSRF queries and invalid MFA codes are no longer passed.

5.0 Residual Risk and Limitations

The lab system is definitely improved but not complete as in a production platform. TOTP secrets are still stored as plain text in the database, not protected by a key that is managed outside of the database. Login and MFA rate tracking is application/session based and not shared per account or per source service. Donation pass tokens are random - there's no user facing expiry or rotation workflow. The Audit Log is not stored in an append-only external logging service, but is stored in the application database. The assignment also takes advantage of Flask's local development server, running on HTTP, as opposed to a production-level WSGI server, HTTPS termination, Secure cookies, and operational monitoring. Finally, neither of the scanners used here can demonstrate that there are no unknown defects in the business logic. Manual review and regression testing are thus still required.

Residual risk	Current control	Recommended future improvement
MFA secret at rest	Secret is access-controlled in PostgreSQL and never shown in reports/logs	Encrypt MFA secret with separately managed application key; define recovery process
Rate limiting	Local login/MFA attempt controls	Centralize per-account/IP throttling with shared storage and alerting
Donation-pass token lifecycle	Random high-entropy token	Add expiry, rotation/revocation and access-history UI
Audit integrity	Structured application audit table	Forward to append-only/centralized logging and alert on suspicious patterns
Transport/deployment	Local lab HTTP and Flask dev server	Deploy behind production WSGI/reverse proxy with HTTPS/HSTS and Secure cookies
Scanner coverage	Seven complementary tools plus manual review	Continue code review, dependency auditing and regression testing in each release

Table 5: Residual risks and recommended future improvements

6.0 Conclusion

The final implementation covers all three parts of Part 2: Secure System Design, Secure coding with explanation and Test Results Analysis. Whereas it revealed how the browser, application, and database interfaces tie off the results of Part 1, it was useful, as several weaknesses could be chained, rather than isolated patches were more convenient, layered controls were more useful. Next up in the code changes were the main causes themselves. Parameterized queries, plaintext passwords were replaced with scrypt hashes, and TOTP MFA, replay protection and rate controls were added to the staff login flow. There is now server-side RBAC and more restrictive data access in the Operational Report. The use of random stored tokens for predictable donation passes has been introduced, unsafe rendering has been eliminated, CSRF protection has been implemented on forms responsible for changing the state, session and response settings have been enhanced, and verbose debug/error leakage has been replaced by safer errors and application level audit logging.

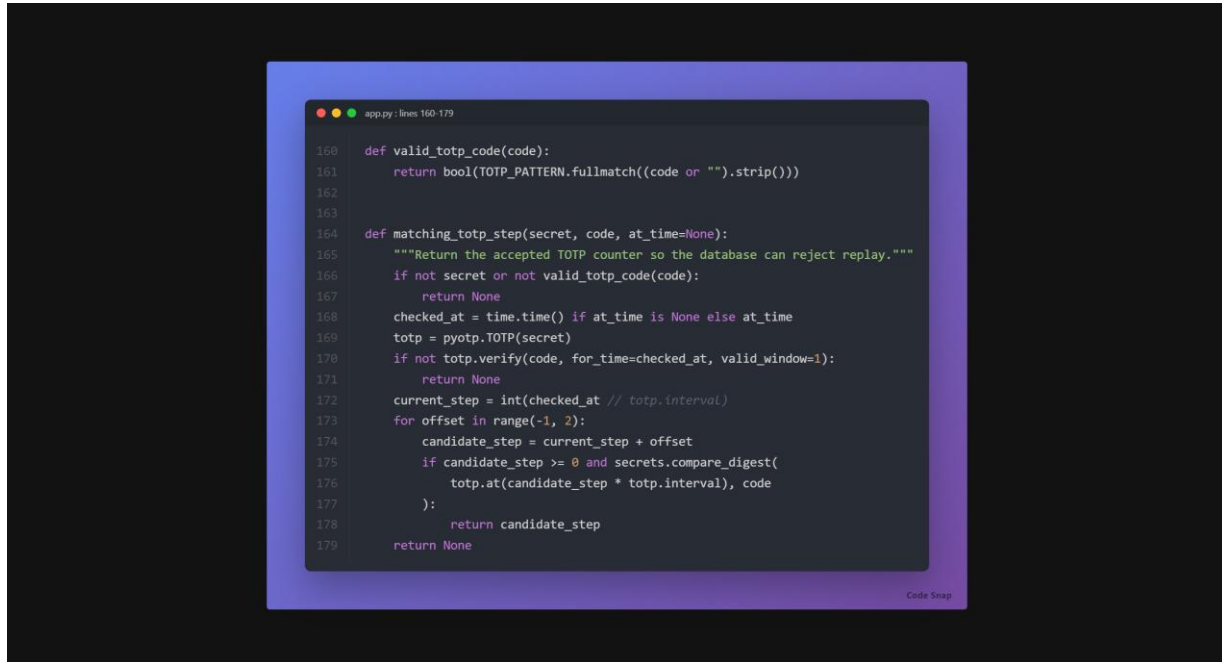
The retest results line up with those changes. A focused Semgrep run reported no matches; Bandit reported none of the Medium or High issues; sqlmap no longer validated either of the original injection paths; and ZAP reported hardened cookie behaviour, SQLi rejection and CSRF enforcement, as well as the MFA step. This was followed by doing dependency upgrades, which was followed by pip-audit reporting no known vulnerabilities and by a successful run of the regression suite by pytest and an observation of the added security headers by Nikto. The term

low risk and limits for the environment are not omitted. That's why it's not just a visual refresh of the major security controls; they can be tracked from a recognized threat to a code change and then to a test result.

The approximate word count of the main text (excluding cover, contents, tables, captions, references and appendix): approximately 4,181 words.

Appendix A Additional Implementation Evidence

Supporting evidence for the secure code and regression is presented in the Appendix, which is important for auditability purposes but distracts from the main story of the implementation if included earlier. The figures represent a single local implementation evidence set.



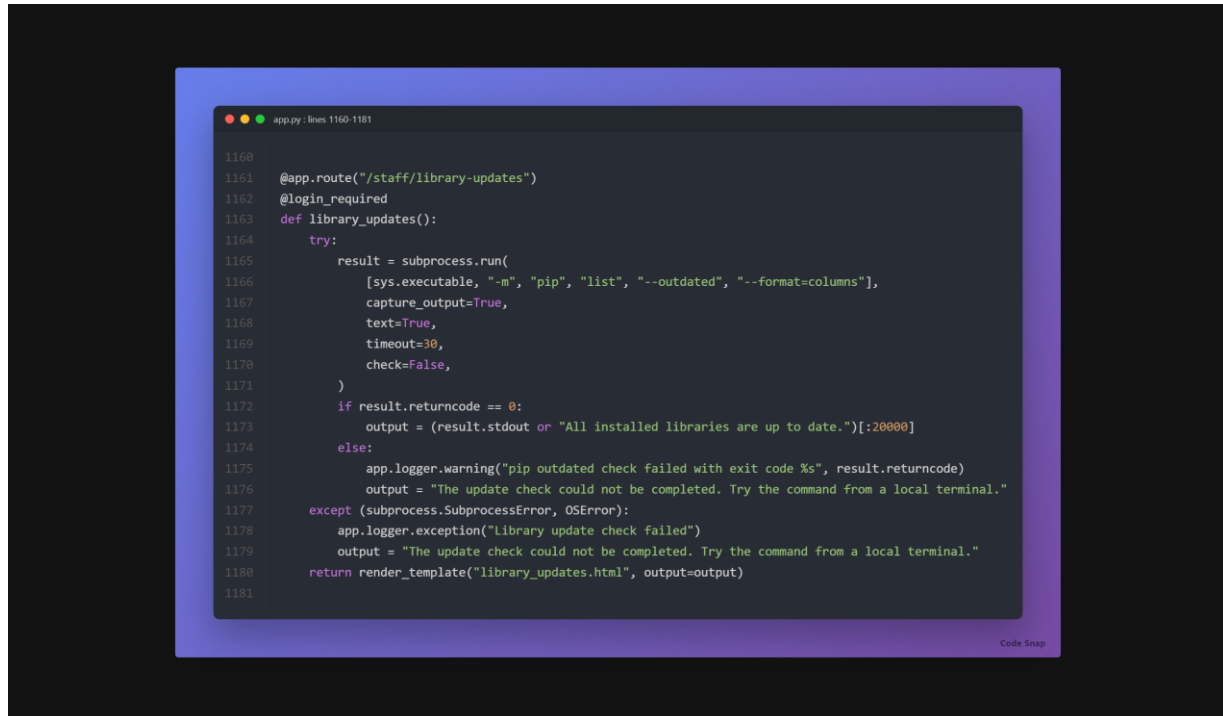
Appendix Figure A1: TOTP time-step validation helper

```

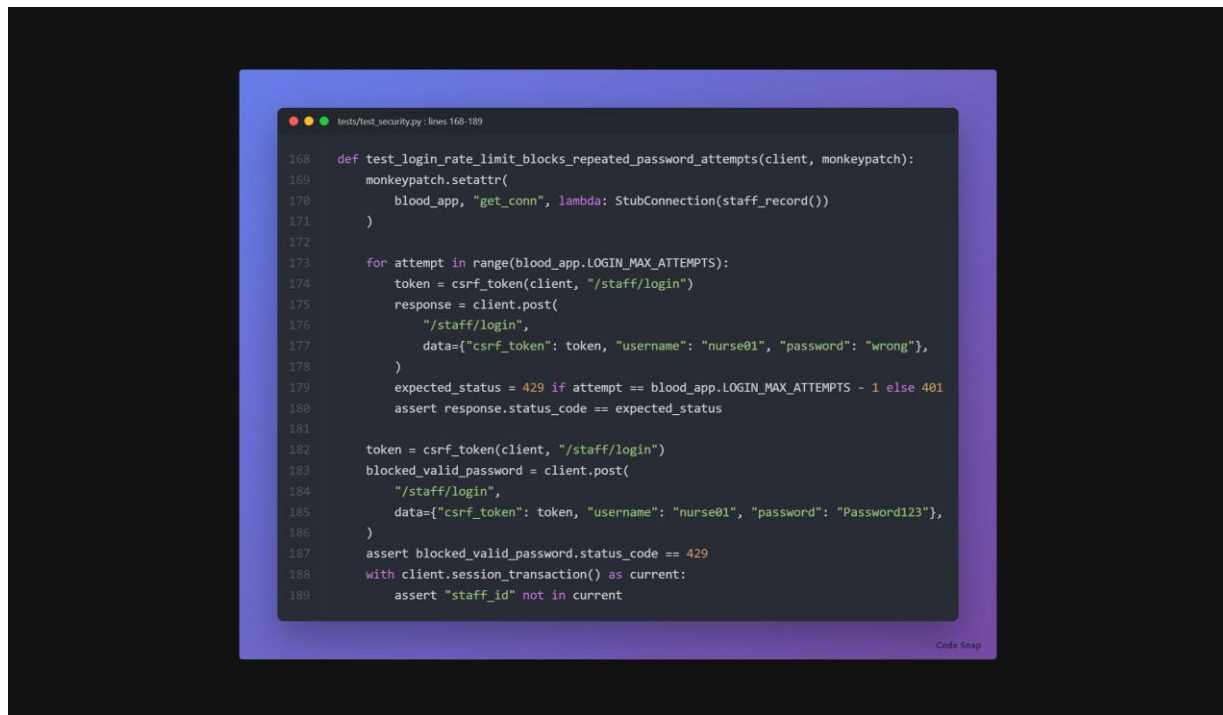
679 def mfa_disable():
680     password = request.form.get("password") or ""
681     code = (request.form.get("code") or "").strip()
682     conn = get_conn()
683     cur = conn.cursor()
684     try:
685         cur.execute(
686             """
687             SELECT id, username, password_hash, mfa_enabled, mfa_secret,
688                mfa_last_used_step
689             FROM medical_staff WHERE id = %s
690             """,
691             (session["staff_id"],),
692         )
693         staff = cur.fetchone()
694         valid_password = bool(staff and check_password_hash(staff["password_hash"], password))
695         totp_step = matching_totp_step(staff["mfa_secret"], code) if staff else None
696         valid_code = bool(
697             staff
698             and staff["mfa_enabled"]
699             and totp_step is not None
700             and (
701                 staff.get("mfa_last_used_step") is None
702                 or totp_step > staff["mfa_last_used_step"]
703             )
704         )
705         if not valid_password or not valid_code:
706             record_security_event(
707                 "mfa_disable",
708                 "failure",
709                 staff_id=staff.get("id") if staff else session.get("staff_id"),
710                 username=staff.get("username") if staff else session.get("username"),
711                 details="reauthentication_failed",
712             )
713             flash("Current password and a valid authenticator code are required.", "danger")
714             return redirect(url_for("mfa_setup"))
715
716         cur.execute(
717             """
718             UPDATE medical_staff
719             SET mfa_enabled = FALSE, mfa_secret = NULL, mfa_confirmed_at = NULL,
720                mfa_last_used_step = NULL
721             WHERE id = %s
722                AND mfa_enabled = TRUE
723                AND mfa_secret IS NOT NULL
724                AND (mfa_last_used_step IS NULL OR mfa_last_used_step < %s)
725             RETURNING id
726             """,
727             (staff["id"], totp_step),
728         )
729         if cur.fetchone() is None:
730             conn.rollback()
731             record_security_event(
732                 "mfa_disable",
733                 "failure",
734                 staff_id=staff["id"],
735                 username=staff["username"],
736                 details="totp_replay_rejected",
737             )
738             flash("Current password and a valid authenticator code are required.", "danger")
739             return redirect(url_for("mfa_setup"))

```

Appendix Figure A2: MFA disable flow requiring re-authentication



Appendix Figure A3: Controlled library update checker



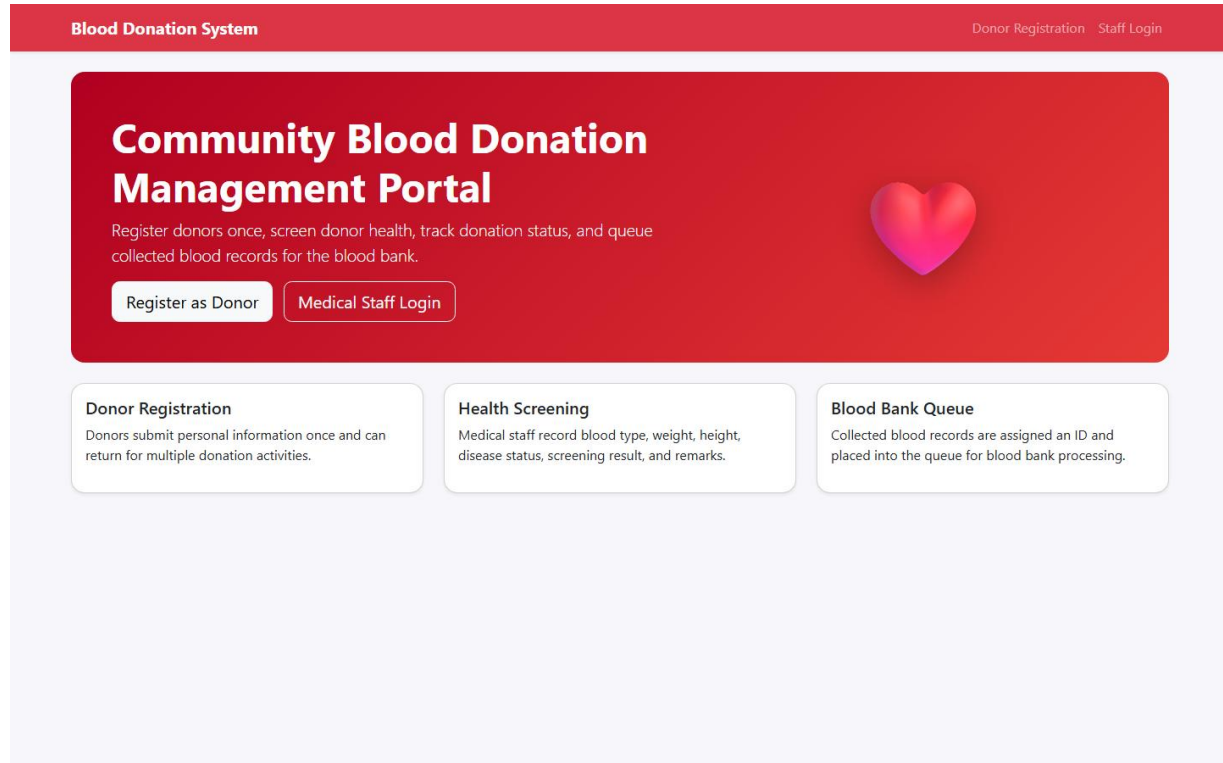
Appendix Figure A4: Login rate-limit regression test

```

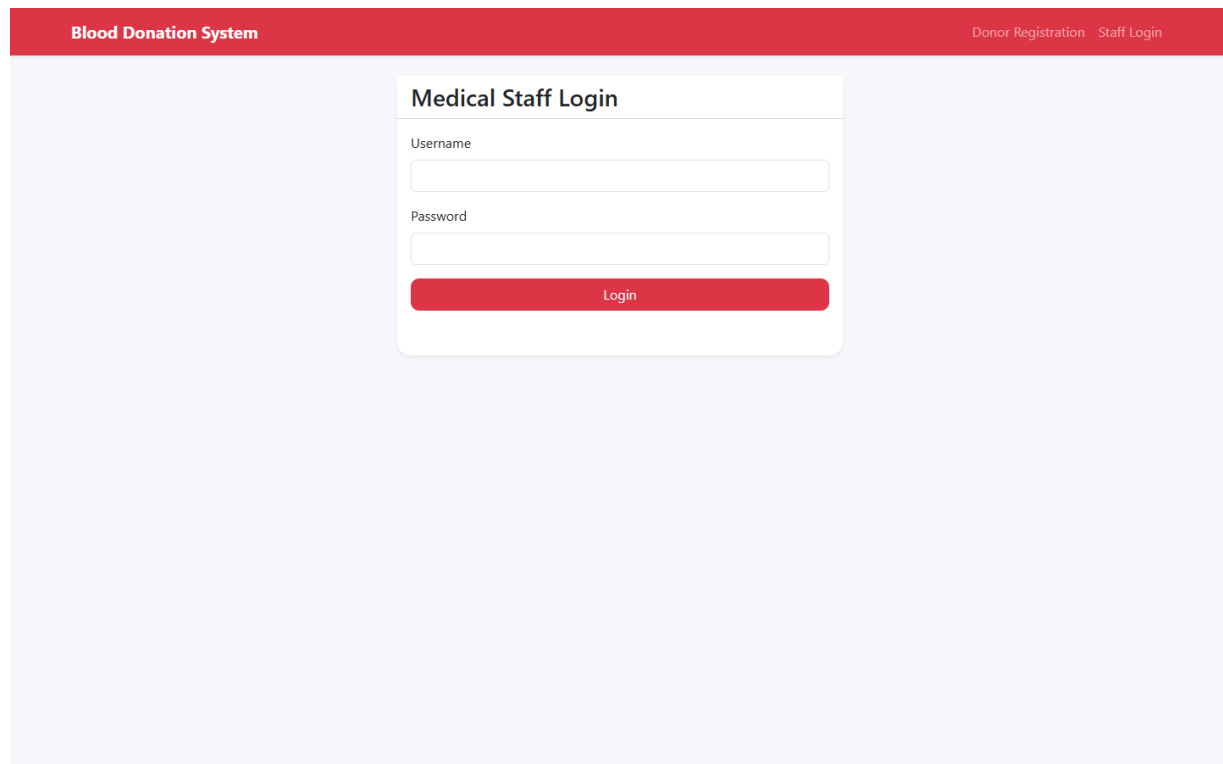
583 def test_used_totp_step_cannot_be_replayed(client, monkeypatch):
584     secret = pyotp.random_base32()
585     staff = staff_record(mfa_enabled=True, mfa_secret=secret)
586     first_claim = StubConnection(staff, {"id": 1})
587     replayed_claim = StubConnection(staff, None)
588     monkeypatch.setattr(
589         blood_app, "get_conn", ConnectionFactory(first_claim, replayed_claim)
590     )
591     current_code = pyotp.TOTP(secret).now()
592
593     with client.session_transaction() as current:
594         current["pending_mfa_staff_id"] = 1
595         current["mfa_attempts"] = 0
596         token = csrf_token(client, "/staff/mfa/verify")
597         accepted = client.post(
598             "/staff/mfa/verify", data={"csrf_token": token, "code": current_code}
599         )
600         assert accepted.status_code == 302
601
602     with client.session_transaction() as current:
603         current.clear()
604         current["pending_mfa_staff_id"] = 1
605         current["mfa_attempts"] = 0
606         token = csrf_token(client, "/staff/mfa/verify")
607         rejected = client.post(
608             "/staff/mfa/verify", data={"csrf_token": token, "code": current_code}
609         )
610         assert rejected.status_code == 401
611         assert "mfa_last_used_step < %s" in replayed_claim.executions[1][0]
612     with client.session_transaction() as current:
613         assert "staff_id" not in current
614
615 def test_audit_event_uses_parameters_and_excludes_authentication_secrets(
616     client, monkeypatch
617 ):
618     connection = StubConnection()
619     monkeypatch.setitem(blood_app.app.config, "AUDIT_DATABASE_ENABLED", True)
620     monkeypatch.setattr(blood_app, "get_conn", lambda: connection)
621     malicious_username = "manager01"; DROP TABLE donors; --"
622
623     with blood_app.app.test_request_context("/staff/login", environ_base={"REMOTE_ADDR": "127.0.0.1"}):
624         blood_app.record_security_event(
625             "staff_login",
626             "failure",
627             username=malicious_username,
628             details="invalid_credentials",
629         )
630
631     query, params = connection.executions[0]
632     assert malicious_username not in query
633     assert params[2] == malicious_username
634     assert params[5] == "invalid_credentials"
635     assert "password" not in query.lower()
636     assert "mfa_secret" not in query.lower()
637     assert connection.committed is True
638
639 def test_schema_supports_totp_replay_prevention_and_minimal_audit_log():
640     schema = Path(blood_app.__file__).with_name("schema.sql").read_text(encoding="utf-8")
641     assert "mfa_last_used_step BIGINT NULL" in schema

```

Appendix Figure A5: TOTP replay and audit regression tests



Appendix Figure A6: Secure application home page



Appendix Figure A7: Secure staff login interface

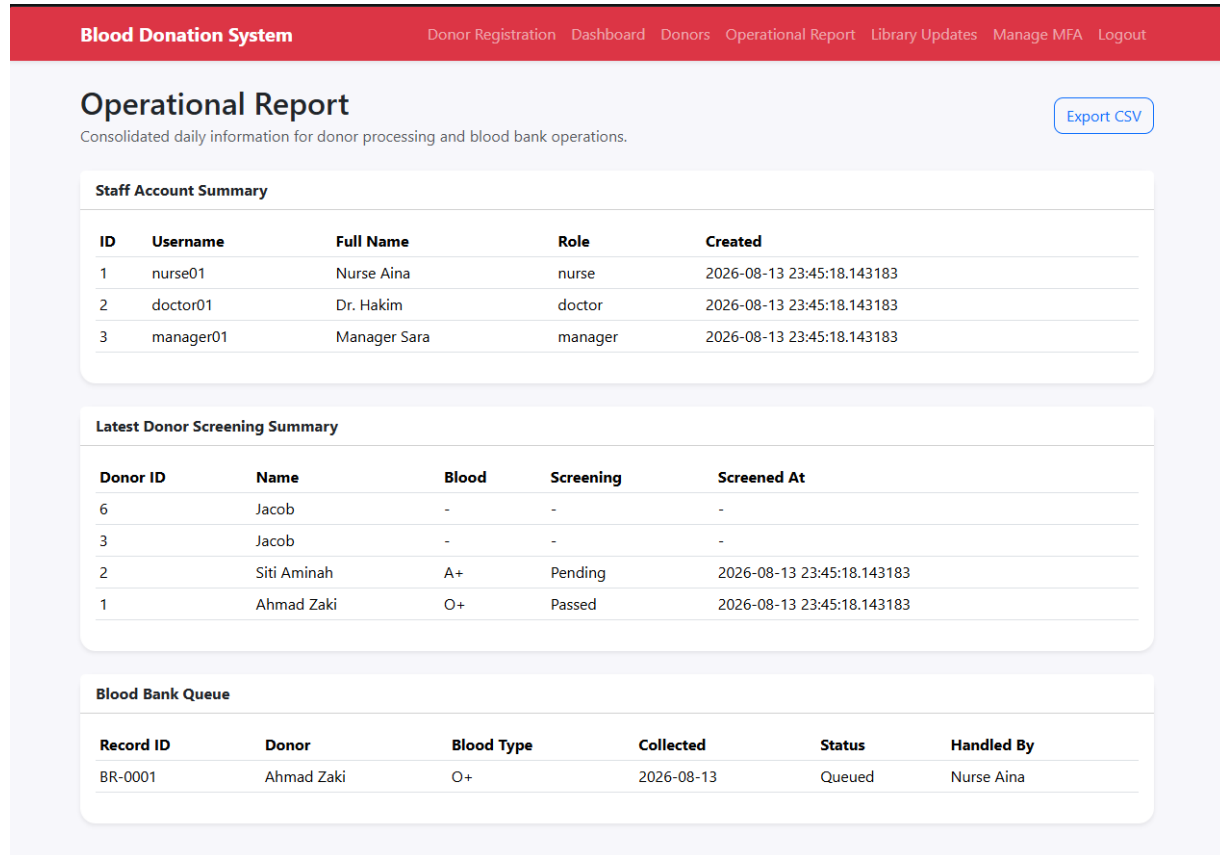
The screenshot shows the 'Blood Donation System' header with links for 'Donor Registration' and 'Staff Login'. A red error message at the top states 'Invalid username or password.' Below this is the 'Medical Staff Login' form, which includes fields for 'Username' and 'Password', and a red 'Login' button.

Appendix Figure A8: Browser evidence that SQL injection login is rejected

The screenshot shows the 'Blood Donation System' header with links for 'Donor Registration', 'Dashboard', 'Donors', 'Library Updates', 'Manage MFA', and 'Logout'. The main section is titled 'Registered Donors' and includes a 'New Donor' button. A search bar contains the text 'Ahmad' and a red 'Search' button. Below the search bar is a table with the following data:

ID	Name	IC / Passport	Email	Phone	Action
1	Ahmad Zaki	900101-10-1234	ahmad@example.com	0123456789	View

Appendix Figure A9: Normal donor search as an authorized nurse



Appendix Figure A10: Secure operational report after role and data-minimisation changes

```

398     if not staff["mfa_enabled"] and not staff["mfa_secret"]:
399         staff["mfa_secret"] = pyotp.random_base32()
400         cur.execute(
401             "UPDATE medical_staff SET mfa_secret = %s WHERE id = %s",
402             (staff["mfa_secret"], staff["id"]),
403         )
404         conn.commit()
405
406     if request.method == "POST" and not staff["mfa_enabled"]:
407         code = (request.form.get("code") or "").strip()
408         if valid_totp_code(code) and pyotp.TOTP(staff["mfa_secret"]).verify(code, valid_window=1):
409             cur.execute(
410                 """
411                 UPDATE medical_staff
412                 SET mfa_enabled = TRUE, mfa_confirmed_at = CURRENT_TIMESTAMP
413                 WHERE id = %s
414                 """
415                 ,
416                 (staff["id"],),
417             )
418             conn.commit()
419             flash("MFA has been enabled for your account.", "success")
420             return redirect(url_for("mfa_setup"))
421             flash("Invalid or expired authenticator code. MFA was not enabled.", "danger")
422
423     if staff["mfa_enabled"]:
424         return render_template("mfa_setup.html", mfa_enabled=True)
425
426     uri = pyotp.TOTP(staff["mfa_secret"]).provisioning_uri(
427         name=staff["username"], issuer_name="Blood Donation System"
428     )
429     qr_image = qrcode.make(uri)
430     buffer = io.BytesIO()
431     qr_image.save(buffer, format="PNG")
432     qr_data = base64.b64encode(buffer.getvalue()).decode("ascii")
433     return render_template(
434         "mfa_setup.html",
435         mfa_enabled=False,
436         qr_data=qr_data,
437         manual_key=staff["mfa_secret"],
438     )

```

Appendix Figure A11: Code evidence for MFA provisioning secret and QR generation

```

358     verified = bool(
359         staff
360         and staff["mfa_enabled"]
361         and staff["mfa_secret"]
362         and valid_totp_code(code)
363         and pyotp.TOTP(staff["mfa_secret"]).verify(code, valid_window=1)
364     )
365     if verified:
366         start_staff_session(staff)
367         flash("MFA verification successful.", "success")
368         return redirect(url_for("staff_dashboard"))
369

```

Appendix Figure A12: Code evidence for MFA code verification

```

7
8 CREATE TABLE medical_staff (
9     id SERIAL PRIMARY KEY,
10    username VARCHAR(50) UNIQUE NOT NULL,
11    password_hash VARCHAR(255) NOT NULL,
12    full_name VARCHAR(150) NOT NULL,
13    role VARCHAR(50) NOT NULL DEFAULT 'nurse'
14    CHECK (role IN ('nurse', 'doctor', 'manager', 'admin')),
15    mfa_enabled BOOLEAN NOT NULL DEFAULT FALSE,
16    mfa_secret TEXT,
17    mfa_confirmed_at TIMESTAMP NULL,
18    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
19 );

```

Appendix Figure A13: MFA-related database fields

Blood Donation System

[Donor Registration](#)[Dashboard](#)[Donors](#)[Library Updates](#)[Manage MFA](#)[Logout](#)

Registered Donors

New Donor

Search by name or IC number

Search

ID	Name	IC / Passport	Email	Phone	Action
2	Siti Aminah	920202-08-5678	siti@example.com	0134567890	View
1	Ahmad Zaki	900101-10-1234	ahmad@example.com	0123456789	View

Appendix Figure A14: Registered donors page after secure changes

7.0 References

Flask-WTF. (2026). CSRF protection. <https://flask-wtf.readthedocs.io/en/latest/csrf/>

Microsoft. (2026). Threat modeling and the STRIDE approach. Microsoft Learn. <https://learn.microsoft.com/en-us/windows-hardware/drivers/driversecurity/threat-modeling-for-drivers>

National Institute of Standards and Technology. (2022). Secure Software Development Framework (SSDF) Version 1.1: Recommendations for Mitigating the Risk of Software Vulnerabilities (NIST SP 800-218). <https://doi.org/10.6028/NIST.SP.800-218>

OWASP Foundation. (2025a). OWASP Top 10:2025. <https://top10.owasp.org/2025/>

OWASP Foundation. (2025b). A01:2025 - Broken Access Control. https://top10.owasp.org/2025/A01_2025-Broken_Access_Control/

OWASP Foundation. (2025c). A07:2025 - Authentication Failures. https://top10.owasp.org/2025/A07_2025-Authentication_Failures/

OWASP Foundation. (2025d). A10:2025 - Mishandling of Exceptional Conditions. https://top10.owasp.org/2025/A10_2025-Mishandling_of_Exceptional_Conditions/

OWASP Foundation. (n.d.-a). Threat Modeling Cheat Sheet. https://cheatsheetseries.owasp.org/cheatsheets/Threat_Modeling_Cheat_Sheet.html

OWASP Foundation. (n.d.-b). SQL Injection Prevention Cheat Sheet. https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html

OWASP Foundation. (n.d.-c). Password Storage Cheat Sheet. https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html

OWASP Foundation. (n.d.-d). Insecure Direct Object Reference Prevention Cheat Sheet. https://cheatsheetseries.owasp.org/cheatsheets/Insecure_Direct_Object_Reference_Prevention_Cheat_Sheet.html

OWASP Foundation. (n.d.-e). Cross Site Scripting Prevention Cheat Sheet. https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html

OWASP Foundation. (n.d.-f). Session Management Cheat Sheet.
https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html

OWASP Foundation. (n.d.-g). Error Handling Cheat Sheet.
https://cheatsheetseries.owasp.org/cheatsheets/Error_Handling_Cheat_Sheet.html

OWASP Foundation. (n.d.-h). Logging Cheat Sheet.
https://cheatsheetseries.owasp.org/cheatsheets/Logging_Cheat_Sheet.html

OWASP Foundation. (n.d.-i). Cross-Site Request Forgery Prevention Cheat Sheet.
https://cheatsheetseries.owasp.org/cheatsheets/Cross-Site_Request_Forgery_Prevention_Cheat_Sheet.html

Pallets Projects. (2026a). Werkzeug security helpers.
<https://werkzeug.palletsprojects.com/en/stable/utils/#module-werkzeug.security>

Pallets Projects. (2026b). Flask security considerations. <https://flask.palletsprojects.com/web-security/>

PyCQA. (2026). Bandit: A security linter for Python. <https://github.com/PyCQA/bandit>

PyOTP. (n.d.). PyOTP documentation. <https://pyauth.github.io/pyotp/>

Python Packaging Authority. (2026). pip-audit. <https://github.com/pypa/pip-audit>

pytest developers. (2026). pytest documentation. <https://docs.pytest.org/en/stable/>

Semgrep. (n.d.). Local and CLI scans. <https://semgrep.dev/docs/category/local-and-cli-scans>

sqlmap project. (2026). Usage. sqlmap Wiki.
<https://github.com/sqlmapproject/sqlmap/wiki/usage>

Sullo, C., & Lodge, D. (2026). Nikto vulnerability scanner. <https://github.com/sullo/nikto>

ZAP by Checkmarx. (2026). Passive scanning and proxy analysis.
<https://www.zaproxy.org/docs/desktop/start/features/pscan/>